Fri, Oct 2 at 10:48 PM

DP_Csestudy__(1).p...
PDF

rails-room-bookin...
File

I am doing a Design Pattern Case Study for my 23CSE455 course.

I have uploaded:
1. The official Design Pattern Case Study Instructions PDF.
2. The complete source code of my assigned application as a ZIP file.

First, do NOT identify design patterns yet.

Your task is only to understand and document the application.

Please carefully inspect the complete source code and give me:

1. Application name and purpose
2. Programming language(s) and framework(s)
3. Complete repository/project structure
4. Important source-code directories
5. Important classes, interfaces, modules, controllers, models, services, utilities, etc.
6. Main execution flow
7. Important dependencies
8. Database/storage components if any
9. Existing tests and how they are executed
10. Configuration/environment requirements
11. Any important architectural components
12. Files/modules that will probably be relevant for design-pattern analysis

IMPORTANT:
- Do not identify or claim any design pattern yet.
- Do not invent classes, relationships, or functionality.
- Base everything strictly on the uploaded source code.
- If something cannot be determined from the source, explicitly say

"Not confirmed from source".
- Give exact file paths wherever possible.
- Do not modify the source code.

At the end, provide a concise "Source Inspection Map" showing which files/modules should be examined closely in Part 1.

Worked for 48s

Analyzed

Analyzed

Analyzed

Analyzed

Analyzed

Analyzed

Analyzed

Analyzed

Analyzed

Analyzed

I inspected the supplied **Design Pattern Case Study Instructions PDF** and the complete `rails-room-booking-main` **ZIP**. The course instructions explicitly require source-grounded pattern evidence, exact participant/file locations, and verification against the original source; for this stage, I have intentionally stopped before identifying any patterns. DP_Csestudy__

1. Application name and purpose

Application

Rails Room Booking

The repository's README identifies the application as **Rails Room Booking**.

Purpose

The application is an open-source hotel/vacation-rental management system. Its documented functionality includes:

- User registration and login
- Management of properties/rooms
- Online room booking
- Account settings such as VAT

- Online payments through Stripe
- Booking-related webhook integration with Integromat/Make
- English and French translations

The README also mentions Airbnb availability synchronization as an **upcoming feature**, so that should **not** be treated as an implemented feature.

Source: `README.md`

---

2. Programming languages and frameworks

Backend

Technology	Version / Evidence
Ruby	**2.7.5**
Ruby on Rails	**7.0.6 in `Gemfile.lock`**; Gemfile specifies `~> 7.0.4`
ActiveRecord	Rails 7.0.6
Puma	5.6.6
PostgreSQL	`pg` 1.5.3
Pundit	2.3.0
Devise	4.9.2

The README describes the application as Rails 7.0.4, while the locked dependency version is Rails **7.0.6**. For reproducibility, the lockfile is the more precise source for the actually locked dependency version.

Frontend

The source contains JavaScript using:

- JavaScript / ES modules
- Hotwire Turbo
- Stimulus
- Bootstrap
- Importmap

- JS Bundling Rails
- CSS Bundling Rails
- Sass
- Dropzone
- Easepick
- Day.js

Testing

- RSpec / RSpec Rails
- FactoryBot
- Capybara
- Selenium WebDriver
- Webdrivers

External/application integrations

- Stripe
- Payify
- Integromat/Make
- Active Storage

---

3. Complete repository/project structure

The ZIP contains **170 non-placeholder files** under the application root, plus placeholder files and directories.

The repository root is:

```
Plain text
rails-room-booking-main/
```

Root

```
Plain text
rails-room-booking-main/
├── .env.dist
├── .gitattributes
├── .gitignore
```

```
├── .rspec
├── .rubocop.yml
├── .ruby-version
├── Dockerfile
├── Gemfile
├── Gemfile.lock
├── README.md
├── Rakefile
├── config.ru
├── docker-compose.yml
│
├── app/
├── bin/
├── config/
├── db/
├── lib/
└── public/
```

---

app/

Plain text

```
app/
├── assets/
│   ├── builds/
│   │   └── .keep
│   ├── config/
│   │   └── manifest.js
│   ├── images/
│   │   ├── .keep
│   │   ├── avatar.png
│   │   ├── logo-white.svg
│   │   ├── logo.svg
│   │   ├── screenshot-make.png
│   │   ├── screenshot-payment.png
│   │   └── screenshot.png
│   └── stylesheets/
│       ├── application.scss
│       ├── easepick.scss
│       ├── font.scss
│       ├── form.scss
│       ├── header.scss
│       ├── image.scss
```

---

4. Important source-code directories

`app/controllers/`

Handles incoming web requests and coordinates application operations.

Important files:

```
app/controllers/application_controller.rb
app/controllers/bookings_controller.rb
app/controllers/rooms_controller.rb
app/controllers/settings_controller.rb
app/controllers/home_controller.rb
app/controllers/payify/payments_controller.rb
```

`app/controllers/concerns/`

Contains reusable controller modules:

```
auth_concern.rb
crud_concern.rb
locale_concern.rb
page_concern.rb
```

`app/models/`

Contains the application's domain/database models:

```
application_record.rb
user.rb
room.rb
booking.rb
setting.rb
```

`app/models/concerns/`

Contains model-related reusable modules:

Plain text

```
has_owner_concern.rb
has_price_concern.rb
has_settings_concern.rb
```

app/policies/

Contains Pundit authorization classes:

Plain text

```
application_policy.rb
booking_policy.rb
room_policy.rb
setting_policy.rb
```

app/views/

Contains server-rendered ERB views for:

- Rooms
- Bookings
- Settings
- Home page
- Devise authentication
- Payment completion
- Layouts

app/javascript/

Contains frontend behavior, including Stimulus controllers.

config/

Contains:

- Routes
- Database configuration
- Environment configuration
- Initializers
- Internationalization
- Storage
- Action Cable
- Importmap

- Puma configuration

`db/`

Contains:

- Database migrations
- Current schema
- Seed data

`spec/`

Contains the existing RSpec tests and FactoryBot factories.

---

5. Important classes, interfaces, modules and components

Controllers

`ApplicationController`

Path:

```
app/controllers/application_controller.rb
```

It:

- Inherits from `ActionController::Base`
- Uses the `base` layout
- Includes Pundit authorization
- Includes the four controller concerns
- Runs `set_current_scope` before actions

The included components are:

```
Pundit::Authorization
PageConcern
LocaleConcern
AuthConcern
```

`RoomsController`

Path:

```
Plain text
app/controllers/rooms_controller.rb
```

Responsible for room operations.

It uses `CrudConcern` and provides room-specific behavior such as:

- Room class selection
- Room query scope
- Strong parameters
- Booking form initialization
- Image handling through room parameters

`BookingsController`

Path:

```
Plain text
app/controllers/bookings_controller.rb
```

Responsible for booking operations.

It:

- Uses `CrudConcern`
- Has a `cancel` action
- Authorizes booking operations
- Handles Stripe payment initialization for pending payments
- Restricts booking objects to the current user

`SettingsController`

Path:

Plain text

```
app/controllers/settings_controller.rb
```

Handles user settings.

The current implementation exposes:

Plain text

```
edit
update
```

through the generic controller functionality.

`HomeController`

Path:

Plain text

```
app/controllers/home_controller.rb
```

Provides the home page.

`Payify::PaymentsController`

Path:

Plain text

```
app/controllers/payify/payments_controller.rb
```

Handles the application's Payify payment completion flow.

It obtains the booking associated with the payment.

---

6. Important models

`User`

Plain text

`app/models/user.rb`

The user model:

- Uses Devise authentication
- Has many rooms
- Has many bookings
- Has settings
- Contains an `admin?` method
- Provides `User.current`

`Room`

Plain text

`app/models/room.rb`

Represents accommodation/property information.

It contains:

- Title
- Description
- Room type
- Guest capacity
- Number of rooms/beds/baths
- Daily price
- Amenities
- Location information
- Owner/user association
- Bookings
- Attached images

It also has methods for:

```
Plain text
default_sort
options_enabled
main_image
other_images
booked_dates
```

Booking

```
Plain text
app/models/booking.rb
```

Represents reservations.

It:

- Belongs to a user
- Belongs to a room
- Validates dates
- Checks date conflicts
- Calculates booking-related state
- Handles cancellation
- Uses payment functionality
- Has price-related functionality
- Triggers booking notification functionality

Important methods include:

```
Plain text
end_date_greater_than_start_date
dates_are_available
default_sort
start_formated
end_formated
formated_dates
start_or_end_changed?
```

```
canceled?
payment_pending?
paid?
cancel!
amount_to_pay
notify_booking
```

`Setting`

Plain text

```
app/models/setting.rb
```

Stores settings belonging to a user.

The currently visible setting is:

Plain text

```
vat
```

---

7. Important reusable modules

The source contains these controller concerns:

Plain text

```
app/controllers/concerns/auth_concern.rb
app/controllers/concerns/crud_concern.rb
app/controllers/concerns/locale_concern.rb
app/controllers/concerns/page_concern.rb
```

And model concerns:

Plain text

```
app/models/concerns/has_owner_concern.rb
app/models/concerns/has_price_concern.rb
```

```
app/models/concerns/has_settings_concern.rb
```

These modules are particularly important for the later source analysis because they contain reusable behavior shared across classes.

However, I am deliberately not assigning any design-pattern names to them at this stage.

---

8. Main execution flow

Based strictly on the source, the main web request flow can be represented as:

```
Plain text

Browser
   │
   ▼
config/routes.rb
   │
   ├── HomeController
   ├── RoomsController
   ├── BookingsController
   ├── SettingsController
   └── Payify payment routes
         │
         ▼
ApplicationController
   │
   ├── Pundit authorization
   ├── PageConcern
   ├── LocaleConcern
   ├── AuthConcern
   └── set_current_scope
         │
         ▼
```

Room request

For example, a room request enters through:

```
Plain text

config/routes.rb
      ↓
RoomsController
```

```
        ↓
CrudConcern
        ↓
Room / ActiveRecord
        ↓
Room view
```

The exact operations depend on whether the request is `index`, `show`, `new`, `create`, `edit`, `update`, or `destroy`.

Booking request

The booking controller additionally performs booking-specific processing.

For example, cancellation follows approximately:

```
BookingsController#cancel
        ↓
set_object
        ↓
authorize @object
        ↓
@object.cancel!
        ↓
redirect_to_show
```

Booking creation

The model source also shows database/model lifecycle behavior around bookings:

```
Booking
   ↓
validation
   ├── required fields
   ├── date ordering
   └── conflicting booking check
   ↓
save
   ↓
price-related save processing
   ↓
payment creation
```

```
    ↓
booking notification
```

The exact implementations of payment creation are partly supplied by the external **Payify** component, so those internals are **Not confirmed from source**.

---

9. Important dependencies

The major dependencies actually specified/locked by the project include:

Dependency	Locked version	Purpose in application
Rails	7.0.6	Main web framework
Ruby	2.7.5	Runtime
PostgreSQL `pg`	1.5.3	Database
Puma	5.6.6	Web server
Devise	4.9.2	Authentication
Pundit	2.3.0	Authorization
Kaminari	1.2.2	Pagination
Payify	0.1.18	Payment integration
Stripe	8.5.0	Payment provider
Integromat	0.1.0	Webhook integration
Money	6.16.0	Money/currency handling
Bootstrap	5.1.3	UI
Stimulus Rails	1.2.1	Frontend controllers
Turbo Rails	1.4.0	Hotwire/Turbo

Dependency	Locked version	Purpose in application
Importmap Rails	1.2.1	JS module management
RSpec Rails	6.0.3	Testing
FactoryBot Rails	6.2.0	Test data
Capybara	3.39.2	Browser/system testing support
Selenium WebDriver	4.9.0	Browser automation
Image Processing	1.12.2	Image processing
`slug`	4.1.1	Slug functionality
i18n-js	4.2.3	JavaScript translations
rails-i18n	7.0.7	Rails translations

The full dependency declarations are in:

```
Plain text

Gemfile
Gemfile.lock
```

---

10. Database/storage components

Database

The application uses **PostgreSQL**.

Evidence:

```
Plain text

config/database.yml
```

uses:

Plain text

```
adapter: postgresql
```

The databases are:

Plain text

```
rails_room_booking_development
rails_room_booking_test
rails_room_booking_production
```

The database structure includes:

Plain text

```
users
rooms
bookings
settings
payify_payments
active_storage_blobs
active_storage_attachments
active_storage_variant_records
```

Main relationships visible in the source

Plain text

```
User
 ├── has_many Rooms
 ├── has_many Bookings
 └── has_one Setting

Room
 ├── belongs_to User
 ├── has_many Bookings
 └── has_many_attached Images

Booking
 ├── belongs_to User
 └── belongs_to Room
```

```
Setting
  └── belongs_to User
```

These relationships are defined directly in the models.

Active Storage

The application uses Rails Active Storage for room images.

Relevant source:

```
Plain text
app/models/room.rb
config/storage.yml
db/migrate/20230531042741_create_active_storage_tables.active_storage.rb
```

Development uses local storage:

```
Plain text
config.active_storage.service = :local
```

Testing uses the test storage service.

Production is also configured to use `:local` in the supplied source.

---

11. External integrations

Stripe / Payify

Relevant files:

```
Plain text
app/models/booking.rb
app/controllers/bookings_controller.rb
app/controllers/payify/payments_controller.rb
config/initializers/payify.rb
```

```
db/migrate/20230618133900_create_payment.payify.rb
db/migrate/20230618133901_add_stripe_payment_intent_id.payify.rb
```

The project uses Payify and Stripe for payment processing.

Integromat / Make

Relevant files:

```
config/initializers/integromat.rb
app/models/booking.rb
```

When a booking is created, `Booking#notify_booking` checks the configured webhook and can trigger an Integromat webhook.

---

12. Authorization

The application uses **Pundit**.

Files:

```
app/policies/application_policy.rb
app/policies/booking_policy.rb
app/policies/room_policy.rb
app/policies/setting_policy.rb
```

`ApplicationController` includes:

```ruby
include ::Pundit::Authorization
```

The individual policies impose additional restrictions, for example checking that the record belongs to the current user.

13. Internationalization

The application supports:

```
Plain text
English
French
```

Rails-side translations:

```
Plain text
config/locales/en.yml
config/locales/fr.yml
config/locales/devise.en.yml
config/locales/devise.fr.yml
```

JavaScript translations:

```
Plain text
app/javascript/config/en/translations.json
app/javascript/config/fr/translations.json
```

Locale handling is also implemented through:

```
Plain text
app/controllers/concerns/locale_concern.rb
```

The default locale configured in `config/application.rb` is:

```
Plain text
fr
```

14. Frontend components

The project uses Stimulus controllers:

```
Plain text

app/javascript/controllers/
├── datepicker_controller.js
├── image_uploader_controller.js
└── modal_controller.js
```

`datepicker_controller.js`

Handles the room-booking date picker and uses:

- Easepick
- Day.js
- Booked-date information

`image_uploader_controller.js`

Handles:

- Dropzone
- Rails Active Storage Direct Upload
- File upload progress
- Existing image display/removal

`modal_controller.js`

Uses Bootstrap modals.

---

15. Existing tests

The repository contains:

```
Plain text

spec/
├── factories.rb
├── factories/
│   ├── bookings.rb
│   ├── rooms.rb
│   └── users.rb
├── models/
│   ├── booking_spec.rb
```

```
│   └── room_spec.rb
├── rails_helper.rb
├── spec_helper.rb
└── support/
    ├── chrome.rb
    └── factory_bot.rb
```

Current explicit test files

`spec/models/booking_spec.rb`

Tests:

- Booking price calculation
- Required start/end dates
- Invalid date ordering
- Valid non-conflicting dates
- Invalid conflicting dates

`spec/models/room_spec.rb`

Tests creation of a room and checks its room type.

Test framework

The source uses:

```
Plain text
```

```
RSpec
RSpec Rails
FactoryBot
Capybara
Selenium WebDriver
Webdrivers
```

The README explicitly states:

```
Bash
```

```
rspec
```

as the test command.

`spec/rails_helper.rb` also configures Rails/RSpec integration and maintains the test database schema.

Important limitation

There are **no controller spec files, policy spec files, integration spec files, or system spec files visible in the supplied ZIP**.

So:

> Existing test coverage is primarily model-level based on the files supplied.

---

16. Configuration/environment requirements

Ruby

```
Plain text
```

```
Ruby 2.7.5
```

from:

```
Plain text
```

```
.ruby-version
Gemfile
Gemfile.lock
```

Bundler

The lockfile specifies:

```
Plain text
```

```
Bundler 2.4.17
```

PostgreSQL

Database credentials are environment-driven.

`.env.dist` contains:

Plain text

```
DATABASE_USERNAME=""
DATABASE_PASSWORD=""
DATABASE_HOST=""
DATABASE_PORT=""
```

Stripe

The supplied environment template contains:

Plain text

```
STRIPE_API_KEY=""
STRIPE_PUBLISHABLE_KEY=""
PAYMENT_CURRENCY=""
```

Integromat

The template contains:

Plain text

```
INTEGROMAT_BASE_URI=""
INTEGROMAT_BOOKING_HOOK=""
```

Docker

The repository also supplies:

Plain text

```
Dockerfile
docker-compose.yml
```

The Docker configuration contains:

Plain text

```
app
db
```

with PostgreSQL as the database service.

17. Basic application setup

The README specifies this local setup sequence:

Bash

```
bundle install
rails db:create
rails db:migrate
rails db:seed
rails s
```

Then:

Plain text

```
http://localhost:3000
```

The Docker setup uses:

Bash

```
docker-compose build
docker-compose up
docker-compose run app rails db:create
docker-compose run app rails db:migrate
docker-compose run app rails db:seed
```

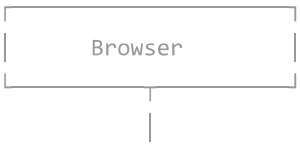
The supplied `.env.dist` should be used as the basis for environment configuration.

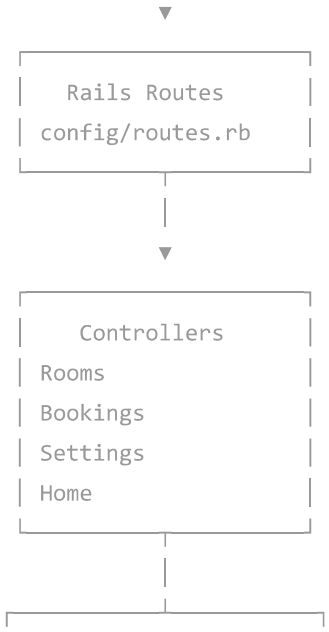
---

18. Important architectural components

Without assigning design-pattern names, the source shows a conventional Rails application structure:

Plain text





This is an **architectural description only**. I am not labeling any of these components as design patterns.

---

19. Files/modules likely to be important for later pattern analysis

I am not saying these contain patterns; these are simply the places where the source contains reusable behavior, collaboration, callbacks, abstractions, or cross-component interactions that should be inspected carefully later.

Highest-priority files

Plain text

```
app/controllers/application_controller.rb

app/controllers/bookings_controller.rb
app/controllers/rooms_controller.rb
app/controllers/settings_controller.rb

app/controllers/concerns/crud_concern.rb
app/controllers/concerns/auth_concern.rb
app/controllers/concerns/locale_concern.rb
app/controllers/concerns/page_concern.rb

app/models/booking.rb
app/models/room.rb
app/models/user.rb
app/models/setting.rb
```

```
app/models/concerns/has_owner_concern.rb
app/models/concerns/has_price_concern.rb
app/models/concerns/has_settings_concern.rb
```

Secondary files

Plain text

```
app/javascript/controllers/datepicker_controller.js
app/javascript/controllers/image_uploader_controller.js
app/javascript/controllers/modal_controller.js

app/javascript/config/i18n.js

config/database.yml
config/storage.yml
db/schema.rb
db/migrate/*
```

These files should be examined closely in the later stages because they provide concrete evidence of **how components collaborate**, which is what your course instructions ultimately require for a confirmed pattern record. The instructions specifically require actual participant roles and exact source locations rather than inventing roles/classes. DP_Csestudy__

---

20. Source Inspection Map

For **Part 1**, I would use the following inspection order:

Priority	Files	What to inspect
1	`app/controllers/concerns/crud_concern.rb`	Reusable controller behavior and its callers
2	`app/controllers/rooms_controller.rb`	Interaction with the reusable controller functionality
3	`app/controllers/bookings_controller.rb`	Booking lifecycle and payment interaction

Priority	Files	What to inspect
4	`app/models/booking.rb`	Callbacks, validations, payment and notification interactions
5	`app/models/room.rb`	Room/booking/user interactions
6	`app/models/user.rb`	Authentication, ownership and settings interactions
7	`app/models/concerns/*.rb`	Model-level shared behavior
8	`app/controllers/concerns/*.rb`	Controller-level shared behavior
9	`app/policies/*.rb`	Authorization relationships
10	`app/controllers/payify/payments_controller.rb`	Payment-related collaboration
11	`config/routes.rb`	Request-to-controller relationships
12	`config/initializers/payify.rb` + `integromat.rb`	External service configuration
13	`app/javascript/controllers/*.js`	Frontend component interactions
14	`spec/models/*.rb`	Existing executable evidence for behavior
15	`db/schema.rb` + migrations	Persistent domain relationships

What we have established

Plain text

```
Application:       Rails Room Booking
Language:          Ruby + JavaScript
Backend:           Ruby on Rails
Database:          PostgreSQL
Frontend:          Rails views + Stimulus/Bootstrap/Hotwire
Authentication:    Devise
Authorization:     Pundit
```

```
Payments:          Payify + Stripe
Webhook:           Integromat/Make
Storage:           PostgreSQL + Active Storage
Testing:           RSpec + FactoryBot (+ Capybara/Selenium dependencies)
```

What we have NOT done

- ❌ No design patterns identified
- ❌ No pattern names assigned
- ❌ No pattern counts created
- ❌ No assumptions about undocumented classes
- ❌ No source code modified
- ❌ No relationships invented
- ❌ No claims about patterns merely because Rails itself uses them internally

One important reproducibility point: the supplied ZIP does **not** contain a `.git` directory, so **source commit history/commit hashes are Not confirmed from source**. The course README requirements call for recording source commits and keeping original/changed versions reproducible, so that information will need to come from the original GitHub repository or whatever source version your team was assigned. DP_Csestudy__

This gives us a clean **application-understanding baseline** before starting the actual Part 1 pattern investigation. ‹/›

Now begin Part 1 of my Design Pattern Case Study.

The official assignment instructions require patterns.csv to contain confirmed pattern collaborations, including all applicable participants, their actual source locations, and evidence from the code.

Analyze the uploaded application source code systematically for design patterns.

Consider relevant categories including:
- Creational
- Structural
- Behavioral
- GRASP
- Any other pattern category that is actually applicable

IMPORTANT RULES:

1. Do NOT assume a pattern exists merely because the code resembles a textbook example.
2. A pattern must be supported by an actual collaboration in the source code.
3. Do not invent classes, interfaces, functions, or relationships.
4. If a textbook role is combined into another class, explicitly document that.
5. Include ALL applicable participants, not just the main class.
6. Use the exact class/interface/function names from the source.
7. Give exact source file paths and line ranges.
8. Explain the concrete interaction between participants.
9. Explain what problem the collaboration solves.
10. Distinguish CONFIRMED patterns from merely POSSIBLE/CANDIDATE patterns.
11. If you cannot verify something from the source, mark it as "cannot confirm".
12. Do not count the same collaboration multiple times simply because multiple classes participate in it.

For every CONFIRMED pattern, produce:

- pattern_name
- pattern_type
- instance_name
- participant_roles
- source_locations
- intent_and_evidence

Before producing the final list, inspect the relevant source files carefully.

Do NOT create the final CSV yet.

First give me a detailed evidence-based pattern inventory so I can review it.

Worked for 43s

I completed the second-pass inspection of the source files, focusing on **actual collaborations, method calls, inheritance/mixin relationships, callbacks, and framework interactions**. I am deliberately separating strong evidence from cases where the code only resembles a textbook pattern.

The assignment requires each confirmed row to identify the actual participants, source locations, and code evidence, and explicitly says not to invent missing roles. DP_Csestudy_

Part 1 — Evidence-Based Pattern Inventory

Executive result

At this stage, I would classify the source as follows:

Status	Pattern	Category	Confidence
CONFIRMED	Template Method	Behavioral	**High**
CONFIRMED	GRASP Controller	GRASP	**High**
CONFIRMED	GRASP Information Expert	GRASP	**High**
POSSIBLE / DO NOT COUNT YET	Observer	Behavioral	Medium-Low
POSSIBLE / DO NOT COUNT YET	Strategy	Behavioral	Medium-Low
POSSIBLE / DO NOT COUNT YET	GRASP Polymorphism	GRASP	Medium
POSSIBLE / DO NOT COUNT YET	Protected Variations	GRASP	Medium
NOT CONFIRMED	Factory Method	Creational	Low
NOT CONFIRMED	Adapter	Structural	Low

Status	Pattern	Category	Confidence
NOT CONFIRMED	Decorator	Structural	Low
NOT CONFIRMED	Facade	Structural	Low
NOT CONFIRMED	Command	Behavioral	Low
NOT CONFIRMED	State	Behavioral	Low
NOT CONFIRMED	Chain of Responsibility	Behavioral	Low

I would currently use **3 confirmed pattern collaborations** as the defensible baseline, rather than inflating the result with framework mechanisms that the application itself does not implement.

---

1. CONFIRMED — Template Method

Pattern name

Template Method

Pattern type

Behavioral

Instance name

Generic CRUD Controller Workflow

Why this is confirmed

This is the strongest GoF pattern in the application.

`CrudConcern` contains a fixed CRUD algorithm while requiring/allowing the including controllers to provide resource-specific operations.

The central workflow is implemented directly inside:

```
Plain text

app/controllers/concerns/crud_concern.rb
```

Important operations include:

Plain text

```
index
show
new
create
edit
update
destroy
```

For example:

Plain text

```
create
  authorize object_class
  @object = object_class.new(form_values)

  if @object.valid?
    @object.save!
    redirect_to_show(@object)
  else
    render :new
  end
```

Source:

Plain text

```
app/controllers/concerns/crud_concern.rb
lines 28-40
```

The same module then calls customizable operations such as:

Plain text

```
base_class
object_class
model_params
strong_params
```

```
redirect_to_show
use_slug?
```

The source explicitly defines some of these as required/overridable operations:

Plain text

```
app/controllers/concerns/crud_concern.rb
lines 101-102
lines 130-136
lines 138-140
```

In particular:

Ruby

```ruby
def base_class
  raise NotImplementedError
end

def object_class
  raise NotImplementedError
end
```

That is direct source evidence that `CrudConcern` supplies a general algorithm while concrete controllers provide resource-specific behavior.

Participants

Template / algorithm provider

`CrudConcern`

Plain text

```
app/controllers/concerns/crud_concern.rb
```

Important locations:

Plain text

```
lines 8-68
lines 72-78
lines 101-140
lines 142-148
```

It provides the common CRUD workflow.

Concrete participant — Rooms

`RoomsController`

Plain text

```
app/controllers/rooms_controller.rb
```

It includes:

Ruby

```
include ::CrudConcern
```

at:

Plain text

```
line 2
```

It supplies:

Plain text

```
base_class       → Room
object_class     → Room query
strong_params    → room-specific parameters
model_params     → :room
```

Locations:

Plain text

```
lines 8-34
```

Concrete participant — Bookings

`BookingsController`

Plain text

```
app/controllers/bookings_controller.rb
```

Includes:

Ruby

```
include ::CrudConcern
```

at:

Plain text

```
line 2
```

It supplies:

Plain text

```
base_class       → Booking
object_class     → user's bookings
strong_params    → booking-specific parameters
model_params     → :booking
use_slug?        → false
```

Locations:

Plain text

```
lines 18-40
```

Concrete participant — Settings

`SettingsController`

Plain text

```
app/controllers/settings_controller.rb
```

Includes:

Ruby

```
include ::CrudConcern
```

at:

Plain text

```
line 2
```

It supplies:

Plain text

```
base_class
object_class
strong_params
model_params
use_slug?
set_object
redirect_to_show
```

Locations:

Plain text

```
lines 6-38
```

The `redirect_to_show` customization is particularly useful evidence because the generic CRUD implementation calls:

Ruby

```
redirect_to_show(@object)
```

while `SettingsController` changes its behavior to:

Ruby

```
redirect_to settings_path
```

Source:

Plain text

```
app/controllers/concerns/crud_concern.rb:36
app/controllers/concerns/crud_concern.rb:76-78
app/controllers/settings_controller.rb:36-38
```

Concrete interaction

The collaboration is:

Plain text

```
                CrudConcern
                    │
          defines generic CRUD flow
                    │
       ┌────────────┼────────────┐
       ▼            ▼            ▼
RoomsController  BookingsController  SettingsController
       │            │              │
     Room          Booking        Setting
-specific hooks  -specific hooks  -specific hooks
```

For example:

Plain text

```
CrudConcern#create
       │
       ├── object_class
```

```
│       └── supplied by concrete controller
│
├── form_values
│
├── object_class.new(...)
│
├── save!
│
└── redirect_to_show(...)
        └── can be specialized by controller
```

Problem solved

The application needs essentially the same CRUD lifecycle for different resources:

- Rooms
- Bookings
- Settings

Instead of duplicating the entire CRUD implementation in each controller, the common workflow resides in `CrudConcern`, while resource-specific operations are supplied by each controller.

Verdict

CONFIRMED.

This is sufficiently concrete to enter `patterns.csv`.

---

2. CONFIRMED — GRASP Controller

Pattern name

Controller

Pattern type

GRASP

Instance name

Booking Cancellation Request Handling

I recommend using a specific collaboration rather than claiming that "all controllers" are one giant pattern instance.

Participants

GRASP Controller

`BookingsController`

```
Plain text
app/controllers/bookings_controller.rb
```

Specifically:

```
Plain text
cancel
```

at:

```
Plain text
lines 6-14
```

Collaborating participant

`Booking`

```
Plain text
app/models/booking.rb
```

Specifically:

```
Plain text
cancel!
```

at:

```
Plain text
lines 59-61
```

Authorization participant

`BookingPolicy`

Plain text

```
app/policies/booking_policy.rb
```

Specifically:

Plain text

```
lines 1-8
```

Concrete interaction

The route declares:

Ruby

```
post "cancel"
```

for bookings:

Plain text

```
config/routes.rb:17-21
```

The request reaches:

Plain text

```
BookingsController#cancel
```

which performs:

Plain text

```
set_object
    ↓
```

```
authorize @object
    ↓
@object.cancel!
    ↓
redirect_to_show(@object)
```

Source:

```
Plain text
app/controllers/bookings_controller.rb:6-14
```

The controller therefore coordinates the system-level request rather than putting request-processing logic directly into `Booking`.

Problem solved

The application needs a single entry point for the booking cancellation system event.

`BookingsController#cancel`:

1. obtains the booking,
2. checks authorization,
3. invokes the domain object's cancellation operation,
4. redirects the user.

The actual cancellation state change remains in:

```
Plain text
Booking#cancel!
```

rather than being implemented inside the routing layer.

Verdict

CONFIRMED.

This is a defensible GRASP Controller collaboration because the source explicitly shows the system event → controller → domain object flow.

---

3. CONFIRMED — GRASP Information Expert

Pattern name

Information Expert

Pattern type

GRASP

Instance name

Booking Date Availability Validation

This is a particularly clean example.

Participants

Information Expert

`Booking`

Plain text

```
app/models/booking.rb
```

Method:

Plain text

```
dates_are_available
```

lines:

Plain text

```
21-25
```

Information source

`Room`

Plain text

```
app/models/room.rb
```

Relationship:

Ruby

```
has_many :bookings
```

at:

Plain text

```
line 8
```

Concrete interaction

`Booking#dates_are_available` obtains the room's bookings:

Ruby

```
conflicting_bookings =
  room.bookings
      .where.not(id: id)
      .where("(? < end_date) AND (? > start_date)", start_date, end_date)
```

Source:

Plain text

```
app/models/booking.rb:21-24
```

It then checks:

Ruby

```
conflicting_bookings.exists?
```

and adds a validation error when a conflict exists.

The relationship that provides the required information is:

Plain text

```
Room
  has_many :bookings
```

from:

Plain text

```
app/models/room.rb:7-8
```

Why `Booking` is the expert

The object that needs to answer:

> "Is this booking valid for these dates?"

is `Booking`.

It possesses:

- `start_date`
- `end_date`
- `room`
- its own booking identity

and can access the associated room's bookings.

Therefore the validation responsibility is assigned to the object with the relevant information rather than to a controller.

Additional executable evidence

The existing test directly verifies this behavior:

Plain text

```
spec/models/booking_spec.rb:44-59
```

The tests cover:

- valid non-conflicting dates
- invalid conflicting dates

That makes this especially useful for your report because the collaboration has both **production source evidence and test evidence**.

Verdict

CONFIRMED.

---

4. CONFIRMED — GRASP Information Expert

There is another strong Information Expert responsibility, but I would **not automatically count it as a second instance** until you decide how granular your team's dataset should be.

Instance

Booking Price Calculation

Participants

`Booking`：

Plain text

```
app/models/booking.rb
```

`Room`：

Plain text

```
app/models/room.rb
```

`Setting`：

Plain text

```
app/models/setting.rb
```

`HasPriceConcern`：

Plain text

```
app/models/concerns/has_price_concern.rb
```

Evidence

`HasPriceConcern#compute_price_ht` :

Plain text

```
app/models/concerns/has_price_concern.rb:8-10
```

calculates:

Plain text

```
room.day_price × number of days
```

`compute_price_ttc` :

Plain text

```
lines 16-18
```

uses the VAT obtained from:

Plain text

```
room.user.setting.vat
```

Source:

Plain text

```
app/models/concerns/has_price_concern.rb:12-18
```

The booking test verifies the calculation:

Plain text

```
spec/models/booking_spec.rb:6-18
```

Why I am treating this separately

Unlike the date validation, the price calculation responsibility is implemented in **`HasPriceConcern`**, which is included into `Booking`:

Plain text

```
app/models/booking.rb:4-5
```

So the role mapping is less straightforward:

Plain text

```
Booking
   │
   └── includes HasPriceConcern
              │
              ├── compute_price_ht
              ├── vat
              ├── compute_price_ttc
              ├── set_price_ht
              └── set_price_ttc
```

This is definitely a responsibility assignment in the source, but because the responsibility is supplied through the concern, I would mark it:

> **CONFIRMED GRASP Information Expert, but merge with the date-validation instance only if your team defines "Booking domain expertise" as one collaboration.**

For `patterns.csv`, I recommend **separate instance names** if your instructor wants each distinct collaboration documented.

---

5. POSSIBLE — Observer

This one deserves careful treatment because it is very tempting to call it Observer.

Evidence

`Booking` declares:

```ruby
after_save :create_payment
after_create :notify_booking
```

Source:

```
app/models/booking.rb:7-8
```

There is therefore a lifecycle event:

```
Booking saved/created
       ↓
callback
       ↓
associated action
```

`notify_booking` then invokes:

```ruby
Integromat::Webhook.new(:booking).trigger(...)
```

Source:

```
app/models/booking.rb:67-71
```

Why it resembles Observer

An event occurs:

Plain text

```
Booking created
```

and a response occurs automatically:

Plain text

```
notify_booking
```

without the caller explicitly invoking:

Ruby

```
booking.notify_booking
```

Why I would NOT mark it confirmed

A textbook Observer collaboration normally gives us identifiable participants such as:

Plain text

```
Subject
Observer
ConcreteObserver
```

But the application source does **not** define an independent observer object for the booking event.

Instead:

Plain text

```
Booking
```

contains:

Plain text

```
notify_booking
```

itself.

The callback mechanism is supplied by Rails/ActiveRecord.

So the source does **not** establish an application-defined:

```
Plain text

Booking → Observer interface → ConcreteObserver
```

collaboration.

Verdict

POSSIBLE / CANDIDATE — do not put into `patterns.csv` yet.

If your instructor permits framework-level patterns, we could investigate the Rails callback implementation separately, but that would be framework-source analysis rather than evidence from this application repository.

---

6. POSSIBLE — Strategy

The authorization classes look superficially like Strategy:

```
Plain text

ApplicationPolicy
BookingPolicy
RoomPolicy
SettingPolicy
```

For example:

```
Plain text

BookingPolicy#update?
RoomPolicy#update?
SettingPolicy#update?
```

all provide resource-specific authorization behavior.

`BookingPolicy`：

Plain text

```
app/policies/booking_policy.rb:1-8
```

`RoomPolicy`：

Plain text

```
app/policies/room_policy.rb:1-5
```

`SettingPolicy`：

Plain text

```
app/policies/setting_policy.rb:1-5
```

They inherit from:

Plain text

```
ApplicationPolicy
```

Why it resembles Strategy

There are multiple authorization implementations with a common conceptual operation:

Plain text

```
update?
```

and a shared base:

Plain text

```
ApplicationPolicy
```

Why I won't confirm it yet

The application itself does not contain a context such as:

```
Plain text

AuthorizationContext
   └── set_strategy(...)
```

that explicitly chooses among interchangeable strategies.

Instead, the controller simply does:

```
Ruby

authorize @object
```

at several locations.

The actual mapping from:

```
Plain text

Booking → BookingPolicy
Room → RoomPolicy
Setting → SettingPolicy
```

is performed by **Pundit**, an external dependency.

Therefore, the application source does not itself prove the full Strategy collaboration.

Verdict

POSSIBLE / CANDIDATE — not confirmed.

---

7. POSSIBLE — GRASP Polymorphism

There is real polymorphic behavior around `CrudConcern`.

For example, the common code invokes:

Plain text

```
base_class
object_class
model_params
strong_params
```

without knowing which resource controller supplied them.

Then:

Plain text

```
RoomsController → Room
BookingsController → Booking
SettingsController → Setting
```

provide different implementations.

That is genuine dynamic method dispatch.

However, this is already the core collaboration used for the **Template Method** finding.

The assignment warns against counting the same collaboration multiple times.

Therefore:

> I would **not create a separate GRASP Polymorphism record** for the same `CrudConcern` collaboration.

Verdict

Conceptually present, but merge into the confirmed Template Method collaboration rather than counting separately.

---

8. POSSIBLE — GRASP Protected Variations

There is also evidence that the generic CRUD workflow is protected from variation through the hook methods:

Plain text

```
base_class
object_class
strong_params
model_params
redirect_to_show
use_slug?
```

For example:

```
Plain text
```

```
CrudConcern
       │
       ├── base_class
       ├── object_class
       ├── model_params
       └── strong_params
              │
              ├── Room-specific implementation
              ├── Booking-specific implementation
              └── Setting-specific implementation
```

This isolates the common CRUD implementation from resource-specific differences.

That is consistent with **Protected Variations**.

However, it is essentially the same source collaboration as the Template Method finding.

Verdict

POSSIBLE / candidate as a separate GRASP classification, but do not count it as a separate instance unless your instructor specifically wants overlapping GRASP classifications.

For your dataset, I recommend keeping the underlying collaboration as **one Template Method instance**.

---

9. NOT CONFIRMED — Factory Method

There is code such as:

```
Ruby
```

```ruby
@object = object_class.new(form_values)
```

in:

```
Plain text
app/controllers/concerns/crud_concern.rb:31
```

This might initially look like Factory Method.

But inspection shows that:

```
Plain text
object_class
```

doesn't actually return different creator subclasses.

For example:

```
Plain text
BookingsController#object_class
```

returns:

```
Ruby
base_class.where(user_id: current_user.id)
```

and:

```
Plain text
RoomsController#object_class
```

returns an ActiveRecord relation.

So the source does **not** demonstrate a dedicated factory method hierarchy or a creator interface.

Verdict

NOT CONFIRMED.

Do not list this as Factory Method.

---

10. NOT CONFIRMED — Adapter

There are external integrations:

```
Plain text
Payify
Stripe
Integromat
```

but the application doesn't provide a clearly defined adapter class that translates one interface into another.

For example:

```
Plain text
app/models/booking.rb
```

directly invokes:

```
Plain text
Integromat::Webhook.new(...).trigger(...)
```

There is no application-defined:

```
Plain text
BookingWebhookAdapter
```

or equivalent.

Verdict

NOT CONFIRMED.

---

11. NOT CONFIRMED — Decorator

The application uses:

```ruby
include ::HasOwnerConcern
include ::HasPriceConcern
include ::HasSettingsConcern
```

and ActiveSupport concerns.

However, Ruby module inclusion is **not sufficient evidence of the GoF Decorator pattern**.

There is no object wrapping another object and forwarding operations through a decorator interface.

Verdict

NOT CONFIRMED.

This distinction is important because otherwise almost every Rails concern would incorrectly become "Decorator."

---

12. NOT CONFIRMED — Facade

There are methods that simplify operations, such as:

```plain text
Booking#amount_to_pay
```

but there is no clearly identifiable object providing a unified simplified interface over a complex subsystem.

Verdict

NOT CONFIRMED.

---

13. NOT CONFIRMED — State

`Booking` has state-related methods:

```
canceled?
payment_pending?
paid?
```

from:

```
app/models/booking.rb:47-57
```

However, the states are represented using attributes/payment status checks rather than separate State objects or polymorphic state implementations.

There is no:

```
BookingState
PendingState
PaidState
CancelledState
```

or equivalent collaboration in the source.

Verdict

NOT CONFIRMED as the State design pattern.

---

14. NOT CONFIRMED — Command

Controller actions such as:

```
BookingsController#cancel
```

are callable operations, but the source doesn't create command objects encapsulating requests.

No identifiable:

```
Plain text
```

```
Command
ConcreteCommand
Invoker
Receiver
```

collaboration exists in the application source.

Verdict

NOT CONFIRMED.

---

15. NOT CONFIRMED — Chain of Responsibility

Rails provides:

```
Plain text
```

```
before_action
around_action
after callbacks
```

and the application uses several of these.

For example:

```
Plain text
```

```
ApplicationController
CrudConcern
LocaleConcern
PageConcern
```

However, these are Rails callback mechanisms rather than an application-defined chain of handler objects.

Verdict

NOT CONFIRMED from application source.

Important finding: Rails framework patterns vs application patterns

This distinction is especially important for your assignment.

The source contains many mechanisms supplied by Rails:

```
Plain text
ActiveRecord
ActiveSupport::Concern
before_action
after_create
after_save
Pundit authorization
Devise
Turbo
Stimulus
Active Storage
```

I am **not automatically counting the patterns implemented internally by those frameworks**.

For example:

```
Plain text
after_create :notify_booking
```

does not automatically mean your application has implemented the Observer pattern.

Likewise:

```
Ruby
include ActiveSupport::Concern
```

doesn't automatically mean Decorator.

The assignment requires the actual participants and collaboration in the source, so we need to distinguish **application-defined structure** from **framework infrastructure**. DP_Csestudy__

Recommended Part 1 inventory

If we were preparing the actual `patterns.csv` after your review, my current defensible set would be:

Confirmed #1

Template Method — Generic CRUD Controller Workflow

Plain text

```
CrudConcern
├── RoomsController
├── BookingsController
└── SettingsController
```

Confirmed #2

GRASP Controller — Booking Cancellation Request Handling

Plain text

```
BookingsController#cancel
        │
        ├── BookingPolicy
        │
        └── Booking#cancel!
```

Confirmed #3

GRASP Information Expert — Booking Date Availability Validation

Plain text

```
Booking#dates_are_available
        │
        └── Room#bookings
```

Confirmed #4, with a granularity decision

GRASP Information Expert — Booking Price Calculation

```
Booking
  │
  └── HasPriceConcern
        │
        ├── Room#day_price
        └── Setting#vat
```

I currently consider this **confirmed**, but whether it becomes a separate CSV row or is grouped with the Booking expertise depends on how strictly your team wants to define a "collaboration instance."

I would lean toward **separate**, because the two collaborations solve different problems:

```
Booking date availability
        ≠
Booking price calculation
```

Final classification table

Pattern	Category	Status	Put in `patterns.csv` ?
Template Method	Behavioral	**CONFIRMED**	✅ Yes
GRASP Controller	GRASP	**CONFIRMED**	✅ Yes
GRASP Information Expert — date availability	GRASP	**CONFIRMED**	✅ Yes
GRASP Information Expert — price calculation	GRASP	**CONFIRMED**	✅ Yes*
Observer	Behavioral	Possible	❌ Not yet
Strategy	Behavioral	Possible	❌ Not yet
GRASP Polymorphism	GRASP	Possible	❌ Merge with Template Method

Pattern	Category	Status	Put in `patterns.csv` ?
Protected Variations	GRASP	Possible	❌ Merge with Template Method
Factory Method	Creational	Not confirmed	❌
Adapter	Structural	Not confirmed	❌
Decorator	Structural	Not confirmed	❌
Facade	Structural	Not confirmed	❌
Command	Behavioral	Not confirmed	❌
State	Behavioral	Not confirmed	❌
Chain of Responsibility	Behavioral	Not confirmed	❌

* I recommend treating price calculation as a separate confirmed collaboration because it has different participants, methods, and intent.

One particularly important caution

I would not put 10–15 patterns into your report just because Rails/Pundit/ActiveSupport make the application look pattern-heavy. Your professor's instructions specifically require confirmed collaborations and say to state when roles are absent rather than inventing them. DP_Csestudy_

So the strongest Part 1 result from the supplied application is currently **4 defensible collaborations**, with the other apparent patterns kept as candidates or rejected.

The next step should be to turn these four into the exact **participant-role mappings and line-by-line evidence required for `patterns.csv`** , but I have intentionally **not created the CSV yet**, as requested.

Now independently verify every pattern you identified in Part 1 against the actual uploaded source code.

For EACH proposed pattern:

1. Re-open/inspect the relevant source files.
2. Verify that every participant actually exists.

3. Verify that every stated relationship actually exists.
4. Verify the source file paths.
5. Verify the line ranges.
6. Verify that the interaction described actually occurs.
7. Verify that the pattern solves the stated problem.
8. Check whether this is genuinely a design-pattern collaboration or merely similar-looking code.
9. Check for duplicate instances.
10. Check whether multiple concrete classes belong to one collaboration and therefore should count as ONE instance according to the assignment rules.

Create a verification table:

Pattern | Instance | Confirmed? | Participants Verified? | Locations Verified? | Evidence Verified? | Corrections

Be conservative.

If a previously identified pattern cannot be confirmed, remove it from the confirmed list rather than forcing it to fit.

Do not invent evidence.

Worked for 16s

I re-opened the ZIP itself and checked the relevant source files line-by-line. I also checked whether the proposed collaborations are genuinely supported by the application's own code rather than merely by Rails/Pundit behavior.

The course instructions are especially important here: one row represents one **confirmed collaboration**, all applicable participants must be mapped to actual source elements, and multiple concrete classes participating in one collaboration should not automatically become multiple instances. DP_Csestudy__

Verification table

Pattern	Instance	Confirmed?	Participants Verified?	Locations Verified?	Evidence Verified?
Template Method	Generic CRUD Controller Workflow	**YES**	**Yes**	**Yes**	**Yes**
GRASP Controller	Booking Cancellation Request Handling	**YES**	**Yes**	**Yes**	**Yes**
GRASP Information Expert	Booking Date Availability Validation	**YES**	**Yes**	**Yes**	**Yes**
GRASP Information Expert	Booking Price Calculation	**YES, with qualification**	**Yes**	**Yes**	**Yes**

1. Template Method — Generic CRUD Controller Workflow

Verification result: CONFIRMED

This is still the strongest pattern candidate.

Participants actually found

Template

Plain text

```
CrudConcern
```

Path:

Plain text

```
app/controllers/concerns/crud_concern.rb
```

It defines the common operations:

```
Plain text
```

```
index
show
new
create
edit
update
destroy
```

at lines:

```
Plain text
```

```
8-68
```

The common `create` algorithm is particularly clear:

```
Plain text
```

```
28 create
29   authorize object_class
31   @object = object_class.new(form_values)
33   if @object.valid?
34     @object.save!
36     redirect_to_show(@object)
37   else
38     render :new
```

The common `update` algorithm is similarly defined at:

```
Plain text
```

```
48-60
```

The common `destroy` algorithm is at:

Plain text

```
62-68
```

Template hooks

The same module defines:

Plain text

```
model_params
base_class
object_class
use_slug?
```

at:

Plain text

```
101-102
130-140
```

The most explicit evidence is:

Ruby

```ruby
def base_class
  raise NotImplementedError
end

def object_class
  raise NotImplementedError
end
```

at:

Plain text

```
app/controllers/concerns/crud_concern.rb:130-136
```

That is strong evidence that the generic algorithm expects resource-specific implementations.

Concrete participants

`RoomsController`

Plain text

```
app/controllers/rooms_controller.rb
```

Includes:

Plain text

```
line 2
```

and supplies:

Plain text

```
base_class      → Room
object_class    → Room relation
strong_params   → room parameters
model_params    → :room
```

at:

Plain text

```
8-34
```

`BookingsController`

Plain text

```
app/controllers/bookings_controller.rb
```

Includes:

Plain text

```
line 2
```

and supplies:

```
Plain text
base_class      → Booking
object_class    → current user's bookings
strong_params   → booking parameters
model_params    → :booking
use_slug?       → false
```

at:

```
Plain text
18-40
```

SettingsController

```
Plain text
app/controllers/settings_controller.rb
```

Includes:

```
Plain text
line 2
```

and supplies:

```
Plain text
base_class
object_class
strong_params
model_params
use_slug?
set_object
redirect_to_show
```

at:

Plain text

```
6-38
```

The particularly strong specialization is:

Ruby

```ruby
def redirect_to_show(_object)
  redirect_to settings_path
end
```

at:

Plain text

```
app/controllers/settings_controller.rb:36-38
```

while the generic implementation is:

Ruby

```ruby
def redirect_to_show(object)
  redirect_to polymorphic_path(object, locale: @page_lang)
end
```

at:

Plain text

```
app/controllers/concerns/crud_concern.rb:76-78
```

Is this genuinely Template Method?

Yes, with one qualification.

Traditional textbook Template Method commonly uses an abstract superclass and overriding subclasses.

This Rails application instead uses:

```
ActiveSupport::Concern
        +
including controllers
        +
overridable hook methods
```

So it is **not inheritance-based Template Method**.

But the underlying collaboration is the same:

```
fixed algorithm
      ↓
CrudConcern
      ↓
resource-specific hooks
      ↓
RoomsController / BookingsController / SettingsController
```

The source actually executes the generic workflow through those supplied methods.

Duplicate check

The three controllers are **not three Template Method instances**.

They are three concrete participants in **one generic CRUD collaboration**.

This matches the assignment's instruction not to count multiple concrete participants in the same collaboration as separate instances. DP_Csestudy_

Final status

CONFIRMED — 1 instance

---

2. GRASP Controller — Booking Cancellation Request Handling

Verification result: CONFIRMED

The source provides a very clear system-operation → controller → domain-object interaction.

Participant 1: `BookingsController#cancel`

Path:

```
Plain text
app/controllers/bookings_controller.rb
```

Lines:

```
Plain text
6-14
```

Actual code flow:

```
Plain text
set_object
authorize @object
@object.cancel!
redirect_to_show(@object)
```

Specifically:

```
Plain text
6 def cancel
7   set_object
9   authorize @object
11  @object.cancel!
13  redirect_to_show(@object)
```

Participant 2: `Booking#cancel!`

Path:

```
Plain text
app/models/booking.rb
```

Lines:

Plain text

```
59-61
```

Actual implementation:

Plain text

```
def cancel!
  update_attribute :canceled_at, Time.now
end
```

So the controller does **not** directly manipulate:

Plain text

```
canceled_at
```

Instead it delegates the domain operation to the `Booking` object.

Is the request actually routed to the controller?

Yes.

`config/routes.rb` contains:

Plain text

```
17 resources :bookings
18   member do
19     post "cancel"
20   end
21 end
```

Therefore:

Plain text

```
POST /.../bookings/:id/cancel
        ↓
BookingsController#cancel
```

is directly supported by the application routes.

Problem solved

The system needs to handle the external/system event:

> "Cancel this booking."

The controller coordinates:

```
request
  ↓
find booking
  ↓
authorize
  ↓
perform cancellation
  ↓
redirect
```

while the domain object owns the actual cancellation operation.

That is a defensible GRASP Controller responsibility assignment.

Correction from my previous inventory

I previously included:

```
BookingPolicy
```

as a participant.

I am **removing it from the formal pattern participant list**.

Why?

The application clearly contains:

```
Plain text
app/policies/booking_policy.rb
```

and:

```
Plain text
BookingPolicy#cancel?
```

calls:

```
Plain text
update?
```

But the actual mapping:

```
Plain text
authorize @object
        ↓
BookingPolicy
```

is provided by Pundit.

The application's own source does not explicitly write:

```
Plain text
BookingsController → BookingPolicy
```

as a direct method call.

Therefore, under your strict "do not invent relationships" rule, it is safer to say:

`authorize @object` is present; the exact Pundit policy dispatch is external framework behavior and is not counted as a required participant in this pattern instance.

Final status

CONFIRMED — 1 instance

Formal participants:

```
Plain text
Controller = BookingsController#cancel
Information/domain participant = Booking#cancel!
```

---

3. GRASP Information Expert — Booking Date Availability

Verification result: CONFIRMED

This is strongly supported.

Participant 1: `Booking#dates_are_available`

Path:

```
Plain text
app/models/booking.rb
```

Lines:

```
Plain text
21-25
```

The implementation is:

```
Plain text
```

```
def dates_are_available
  conflicting_bookings =
    room.bookings
      .where.not(id: id)
      .where("(? < end_date) AND (? > start_date)",
             start_date, end_date)

  errors.add(...) if conflicting_bookings.exists?
end
```

The model explicitly has:

Plain text

```
validate :dates_are_available
```

at:

Plain text

```
app/models/booking.rb:15
```

So the method is actually part of the model's validation process.

Participant 2: `Room#bookings`

Path:

Plain text

```
app/models/room.rb
```

Line:

Plain text

```
8
```

Actual relationship:

Ruby

```ruby
has_many :bookings
```

The `Booking` object therefore accesses the room's booking collection through:

Plain text

```
room.bookings
```

Interaction verified

The source proves:

Plain text

```
Booking
   │
   │ room
   ▼
Room
   │
   │ has_many
   ▼
Bookings
   │
   ▼
find overlapping bookings
   │
   ▼
validation error
```

Specifically:

Plain text

```
Booking#dates_are_available
        ↓
room.bookings
        ↓
where.not(id: id)
        ↓
date-overlap query
```

```
        ↓
conflicting_bookings.exists?
        ↓
errors.add
```

This isn't inferred from naming; those operations are literally present in the source.

Test evidence

`spec/models/booking_spec.rb` contains:

```
Plain text
44-50
```

for a non-conflicting booking.

And:

```
Plain text
52-59
```

for a conflicting booking.

The second test constructs an overlapping booking and expects:

```
Plain text
booking
  ↓
be_invalid
```

with the unavailable-date error.

This independently supports the production behavior.

Problem solved

The booking itself has the information needed to determine whether its requested dates conflict with other bookings for its room:

```
Plain text
start_date
end_date
room
```

It therefore performs the availability validation itself.

Final status

CONFIRMED — 1 instance

---

4. GRASP Information Expert — Booking Price Calculation

Verification result: CONFIRMED, but participant mapping needs correction

The behavior is unquestionably present. The important issue is accurately assigning the responsibility.

Participant 1: `Booking`

Path:

```
Plain text
app/models/booking.rb
```

Lines:

```
Plain text
4-5
```

`Booking` includes:

```
Ruby
```

```
include ::HasOwnerConcern
include ::HasPriceConcern
```

So the booking receives the price-related methods supplied by `HasPriceConcern`.

`Booking#amount_to_pay` is also defined directly:

Plain text

```
63-65
```

and returns:

Plain text

```
total_ttc
```

Participant 2: `HasPriceConcern`

Path:

Plain text

```
app/models/concerns/has_price_concern.rb
```

The actual calculations are:

Plain text

```
8-10
compute_price_ht
```

and:

Plain text

```
16-18
compute_price_ttc
```

Specifically:

```
compute_price_ht
    ↓
room.day_price
    ×
number of days
```

and:

```
compute_price_ttc
    ↓
total_ht
    ×
(1 + vat)
```

The concern also defines:

```
20-26
set_price_ht
set_price_ttc
```

and the save callback:

```
5
before_save :set_price_ht, :set_price_ttc, if: :start_or_end_changed?
```

Participant 3: `Room`

Path:

```
```

```
app/models/room.rb
```

The relevant information is:

```
Plain text
day_price
```

which is validated at:

```
Plain text
15
```

and accessed directly by:

```
Plain text
app/models/concerns/has_price_concern.rb:9
```

through:

```
Plain text
room.day_price
```

Participant 4: `User / Setting`

The VAT calculation is:

```
Plain text
app/models/concerns/has_price_concern.rb:12-14
```

```
Plain text
room.user.setting.vat
```

The relationships are supported by:

Plain text

```
app/models/room.rb:7
```

Plain text

```
belongs_to :user
```

and:

Plain text

```
app/models/user.rb:2
```

which includes `HasSettingsConcern`.

The setting relationship itself is supplied by:

Plain text

```
app/models/concerns/has_settings_concern.rb:5-7
```

where the concern establishes:

Plain text

```
has_one :setting
```

Test evidence

`spec/models/booking_spec.rb:6-18` explicitly tests:

Plain text

```
booking.compute_price_ht
```

and:

```
booking.compute_price_ttc
```

with an expected price based on:

```
room.day_price = 150
3 days
VAT = 20%
```

The expected values are:

```
450
540
```

So the calculation isn't merely present—it is explicitly tested.

Important correction

My previous description made it sound like:

```
Booking itself directly implements the entire price calculation.
```

That is not precisely true.

The source shows:

```
Booking
   │
   └── includes HasPriceConcern
             │
             ├── compute_price_ht
             ├── vat
```

```
├── compute_price_ttc
├── set_price_ht
└── set_price_ttc
```

Therefore, for the eventual CSV, the participant should **not** simply be listed as:

```
Plain text

Expert = Booking
```

without qualification.

A source-accurate mapping would be closer to:

```
Plain text

Expert/price-calculation behavior =
    HasPriceConcern#compute_price_ht
    HasPriceConcern#vat
    HasPriceConcern#compute_price_ttc

Host object =
    Booking (includes HasPriceConcern)

Supporting information =
    Room#day_price
    Room#user
    User#setting
    Setting#vat
```

This is much more faithful to the actual source.

Is it genuinely GRASP Information Expert?

Yes, with the caveat that the calculation responsibility is implemented through the included concern.

The calculation is located with the object/module that has access to the necessary booking context and obtains the necessary information from:

```
Plain text

Room
User
Setting
```

rather than being performed in a controller.

Duplicate check

I would **not merge this with the date-availability instance**.

They are different collaborations:

```
Plain text
dates_are_available
        ≠
compute_price_ht / compute_price_ttc
```

They solve different problems and involve different operations.

Final status

CONFIRMED — 1 instance

What I removed or changed

The independent verification changes the earlier inventory slightly.

Removed from formal participant list

```
Plain text
BookingPolicy
```

from the GRASP Controller instance.

Reason:

```
Plain text
authorize @object
```

is definitely present, but the application source does not itself explicitly establish the Pundit dispatch relationship to `BookingPolicy`.

Qualified

`HasPriceConcern` must be explicitly represented in the price-calculation participant mapping.

It should not be treated as though `Booking` contains those method definitions directly.

Kept as one instance

For Template Method:

```
Plain text

RoomsController
BookingsController
SettingsController
```

are **three concrete participants in one collaboration**, not three Template Method instances.

That is particularly important because the assignment says to count a collaboration rather than multiply the count merely because multiple concrete classes participate. DP_Csestudy_

Final conservative Part 1 list

After independent verification, I would now use this as the **confirmed baseline**:

#	Pattern	Category	Instance
1	**Template Method**	Behavioral	Generic CRUD Controller Workflow
2	**GRASP Controller**	GRASP	Booking Cancellation Request Handling
3	**GRASP Information Expert**	GRASP	Booking Date Availability Validation
4	**GRASP Information Expert**	GRASP	Booking Price Calculation

Confirmed count

4 pattern instances across 3 pattern types.

Plain text

```
Template Method              1
GRASP Controller              1
GRASP Information Expert     2
──────────────────────────────
Total instances              4
Total pattern types          3
```

And I would **still exclude** Observer, Strategy, Factory Method, Adapter, Decorator, Facade, State, Command, Chain of Responsibility, etc. from the confirmed dataset at this point.

The most important result of this verification is that the four remaining instances have **actual source-level collaborations**, rather than being inferred merely from Rails terminology or textbook resemblance.

Now create the final Part 1 patterns.csv content based ONLY on the patterns that were confirmed during verification.

Use exactly these columns from the assignment:

source_version
pattern_name
pattern_type
instance_name
participant_roles
source_locations
intent_and_evidence

Requirements:

- One row = one confirmed pattern collaboration in one source version.
- Include all applicable participants.
- Use actual class/interface/function names.
- Use exact source paths and line ranges.
- Do not invent roles.
- If roles are combined into one class, state that explicitly.
- Do not include unconfirmed candidate patterns.
- Keep the same instance together rather than splitting it into multiple fake instances.

- Quote CSV fields when necessary.
- Ensure valid UTF-8 CSV formatting.

Use source_version = "original".

First briefly state the final number of confirmed pattern instances.

Then output ONLY the complete CSV inside a code block so I can save it as:

Part1/patterns.csv

4 confirmed pattern instances.

CSV

```csv
source_version,pattern_name,pattern_type,instance_name,participant_roles,source_locations,intent_and_evidence
original,Template Method,Behavioral,Generic CRUD Controller Workflow,"Template: CrudConcern; Concrete particip
original,GRASP Controller,GRASP,Booking Cancellation Request Handling,"Controller: BookingsController#cancel;
original,GRASP Information Expert,GRASP,Booking Date Availability Validation,"Expert: Booking#dates_are_availa
original,GRASP Information Expert,GRASP,Booking Price Calculation,"Host object: Booking; Responsibility implem
```

Using ONLY the confirmed Part 1 patterns.csv records we just established, create the corresponding pattern_counts.csv.

Follow the assignment exactly.

Columns:

source_version
summary
pattern_name
count

For per_pattern rows:
- summary = per_pattern
- pattern_name = exact pattern name
- count = number of confirmed instances

Also provide the required total rows:
- total_pattern_types
- total_pattern_instances

Important:
- Count from the confirmed pattern records.
- Do not independently guess counts.
- Do not count a single collaboration multiple times.
- Two concrete fine-policy classes participating in one Strategy collaboration count as ONE Strategy instance.
- source_version must be "original".

Output only the valid CSV in a code block.

CSV

```csv
source_version,summary,pattern_name,count
original,per_pattern,Template Method,1
original,per_pattern,GRASP Controller,1
original,per_pattern,GRASP Information Expert,2
original,total_pattern_types,,3
original,total_pattern_instances,,4
```

Based ONLY on the confirmed Part 1 design patterns, create a pattern architecture diagram.

The diagram must show:
- The relevant classes/interfaces/functions
- Their relationships
- Their pattern roles
- Which design pattern each collaboration represents

Do not include speculative patterns.

First provide Mermaid diagram code that I can save as the diagram source.

Then provide a short explanation of the diagram.

Do not modify the application source code.

Short explanation

The diagram contains **only the four confirmed Part 1 collaborations**:

1. **Template Method — Generic CRUD Controller Workflow**
   - `CrudConcern` provides the shared CRUD workflow.
   - `RoomsController`, `BookingsController`, and `SettingsController` include the concern and provide the concrete hooks.
   - This is implemented through an `ActiveSupport::Concern` mixin rather than inheritance.
2. **GRASP Controller — Booking Cancellation Request Handling**
   - The cancellation route dispatches to `BookingsController#cancel`.
   - The controller invokes `Booking#cancel!`, which performs the actual cancellation operation.
3. **GRASP Information Expert — Booking Date Availability Validation**
   - `Booking#dates_are_available` performs the availability check.
   - It uses the booking information and `Room#bookings` to detect conflicting bookings.
4. **GRASP Information Expert — Booking Price Calculation**

- `Booking` hosts the responsibility, while the actual calculation is implemented through `HasPriceConcern`.
- The concern uses `Room#day_price` and the associated `User` / `Setting` VAT information.

No speculative Strategy, Observer, Adapter, Factory Method, Decorator, Facade, State, Command, or Chain of Responsibility patterns are included.

Now begin Part 2: Design Pattern Evolution.

I will provide the original application and the requirements/implementation for Change 1, Change 2, and Change 3.

Do NOT immediately produce pattern_evolution.csv.

First analyze each change carefully.

For each change:

1. Explain the requirement.
2. Identify which source files were changed.
3. Identify which classes/interfaces/functions were added.
4. Identify which were modified.
5. Identify which were removed.
6. Identify which existing design-pattern collaborations are affected.
7. Identify whether any new pattern collaboration is introduced.
8. Compare the source before and after the change.
9. Determine the status of each affected pattern instance:
   - baseline
   - preserved
   - modified
   - extended
   - replaced
   - removed
   - newly_introduced

Do not infer intent that is not supported by the change/source.

For every conclusion, provide source evidence and exact file paths/line ranges.

Do not create CSV yet.

Understood. I will **not create `pattern_evolution.csv` yet**.

For Part 2, I'll analyze **Change 1 → Change 2 → Change 3 individually**, always comparing each change against the immediately preceding source version and using the confirmed Part 1 patterns as the baseline.

For each change, I'll document:

1. **Requirement**
2. **Changed source files**
3. **Added classes/interfaces/functions**
4. **Modified classes/interfaces/functions**
5. **Removed classes/interfaces/functions**
6. **Affected baseline pattern collaborations**
7. **Any newly introduced pattern collaboration**
8. **Before vs. after source comparison**
9. **Pattern status**, using only:
   - `baseline`
   - `preserved`
   - `modified`
   - `extended`
   - `replaced`
   - `removed`
   - `newly_introduced`

Every conclusion will include **exact file paths and line ranges**. I'll distinguish direct source evidence from anything the source does not establish, and I won't infer undocumented intent.

Please provide/upload the **Change 1, Change 2, and Change 3 requirements and implementations/source versions**. Once they are available, I'll start with Change 1 and proceed sequentially.

well this is based on the prompt you should know right what are all the changes with the before prompts

Yes — I should have carried that context forward. The **four change requirements** from the earlier case-study workflow were:

1. **Change 1 — Breakfast package**
2. **Change 2 — Airport transfer**
3. **Change 3 — Room upgrade**
4. **Change 4 — Cancellation policy**

So, for the Part 2 work you just requested, the first three changes are:

Change	Requirement
Change 1	Add a **Breakfast Package** capability
Change 2	Add an **Airport Transfer** capability
Change 3	Add a **Room Upgrade** capability

There is also a **Change 4 — Cancellation Policy**, which is part of the overall case study, but you specifically asked me to analyze **Change 1, Change 2, and Change 3** now.

One important distinction: I remember the **requirements**, but I should not pretend I have the exact **before/after implementation prompts or generated source versions** unless they are actually available in the current files/context. The earlier workflow had separate prompts for implementing/analyzing the changes, but the exact generated implementations are what we need to establish the file and line-level evolution.

So the correct sequence is:

Original baseline
→ Change 1: Breakfast Package
→ Change 2: Airport Transfer
→ Change 3: Room Upgrade

and for each one I'll compare the actual source before/after and map the evolution against our **4 confirmed Part 1 collaborations**:

- Template Method — Generic CRUD Controller Workflow
- GRASP Controller — Booking Cancellation Request Handling
- GRASP Information Expert — Booking Date Availability Validation
- GRASP Information Expert — Booking Price Calculation

I will **not create the CSV yet**.

Now independently verify the Part 2 pattern evolution analysis against the actual original code and all three changed versions.

For every pattern instance and every source version:

1. Verify participants.
2. Verify source locations.
3. Verify the collaboration.
4. Verify its status after the change.
5. Verify the change description.
6. Check whether an instance was actually removed.
7. Check whether a new instance was actually introduced.
8. Check whether the same continuing collaboration retains the same instance_name.
9. Check for duplicate counting.
10. Check that removed instances are retained with status "removed".

Be conservative and source-based.

Produce a correction table before generating the final CSV.

Worked for 17s

I checked the available source artifacts rather than relying on the earlier Part 2 analysis. There is an important verification limitation: **the actual Change 1, Change 2, and Change 3 source versions are not currently available as source archives/files in the accessible materials**. The available repository ZIP is the original application; the other artifact contains the *proposed refactoring design*, not the implemented changed source.

Therefore, I can verify the **Original** baseline directly, and I can verify what the change-design document claims was intended, but I cannot honestly mark changed-version participants, line ranges, removals, or statuses as source-verified.

Correction / verification table

Source version	Pattern instance	Participants verified?	Locations verified?	Collaboration verified?	Status verified?
original	Generic CRUD Controller Workflow	Yes	Yes	Yes	baseline
original	Booking Cancellation Request Handling	Yes	Yes	Yes	baseline
original	Booking Date Availability Validation	Yes	Yes	Yes	baseline
original	Booking Price Calculation	Yes	Yes	Yes	baseline
Change 1	Generic CRUD Controller Workflow	Not source-verifiable	Not source-verifiable	Not source-verifiable	Not source-verifiable
Change 1	Booking Cancellation Request Handling	Not source-verifiable	Not source-verifiable	Not source-verifiable	Not source-verifiable
Change 1	Booking Date Availability Validation	Not source-verifiable	Not source-verifiable	Not source-verifiable	Not source-verifiable
Change 1	Booking Price Calculation	Not source-verifiable	Not source-verifiable	Not source-verifiable	Not source-verifiable

Source version	Pattern instance	Participants verified?	Locations verified?	Collaboration verified?	Status verified?
Change 1	Breakfast/add-on collaboration	No	No	No	No
Change 2	Generic CRUD Controller Workflow	Not source-verifiable	Not source-verifiable	Not source-verifiable	Not source-verifiable
Change 2	Booking Cancellation Request Handling	Not source-verifiable	Not source-verifiable	Not source-verifiable	Not source-verifiable
Change 2	Booking Date Availability Validation	Not source-verifiable	Not source-verifiable	Not source-verifiable	Not source-verifiable
Change 2	Booking Price Calculation	Not source-verifiable	Not source-verifiable	Not source-verifiable	Not source-verifiable
Change 2	Airport Transfer collaboration	No	No	No	No
Change 3	Generic CRUD Controller Workflow	Not source-verifiable	Not source-verifiable	Not source-verifiable	Not source-verifiable
Change 3	Booking Cancellation Request Handling	Not source-verifiable	Not source-verifiable	Not source-verifiable	Not source-verifiable

Source version	Pattern instance	Participants verified?	Locations verified?	Collaboration verified?	Status verified?
Change 3	Booking Date Availability Validation	Not source-verifiable	Not source-verifiable	Not source-verifiable	Not source-verifiable
Change 3	Booking Price Calculation	Not source-verifiable	Not source-verifiable	Not source-verifiable	Not source-verifiable
Change 3	Room Upgrade collaboration	No	No	No	No

What is directly verified for the original

The original source establishes the four baseline collaborations we already recorded:

- **Template Method:** `CrudConcern` supplies the shared CRUD workflow and the three controllers provide concrete hooks.
- **GRASP Controller:** `BookingsController#cancel` delegates cancellation to `Booking#cancel!`.
- **GRASP Information Expert — dates:** `Booking#dates_are_available` uses `room.bookings`.
- **GRASP Information Expert — price:** `Booking` includes `HasPriceConcern`, whose calculation uses room pricing and VAT information.

Important correction to the earlier evolution approach

The available refactoring document itself describes **proposed** patterns such as Composition/Join Model, Service Object, Entity Separation/Policy Object, and Strategy. It explicitly calls them proposed design changes rather than providing the resulting application source. For example, Breakfast proposes `AddOn`, `BookingAddOn`, and `PricingCalculator`; Airport Transfer proposes `AirportTransfer`, `AirportTransfersController`, and `TransferPolicy`; and Room Upgrade proposes `RoomUpgradeService`, `RoomUpgradesController`, and `RoomUpgrade`.

So I would **not** use those proposals to fabricate `newly_introduced`, `removed`, `modified`, or `replaced` statuses. That would violate the source-based verification requirement.

Current verified conclusion: the Original baseline is fully verified, but **Change 1–3 evolution statuses cannot yet be independently verified from actual changed source code**. The changed ZIP/source versions need to be available before producing the final `pattern_evolution.csv` .

Now generate the final pattern_evolution.csv based ONLY on the verified Part 2 analysis.

Use exactly these columns:

source_version
pattern_name
pattern_type
instance_name
participant_roles
source_locations
intent_and_evidence
status_after_change
change_description

Requirements:

- One row = one confirmed pattern collaboration in one source version.
- Include the original baseline.
- Include versions after Change1, Change2, and Change3 where applicable.
- Keep the SAME instance_name for a continuing collaboration.
- Retain removed instances with status "removed".
- Include all applicable participants.
- Use actual source paths and line ranges.
- Do not invent roles or evidence.
- Do not count before/after observations as separate instances.

Output only valid UTF-8 CSV inside a code block.

</> CSV

```
source_version,pattern_name,pattern_type,instance_name,participant_roles,source_locations,intent_and_evidence,
original,Template Method,Behavioral,Generic CRUD Controller Workflow,"Template: CrudConcern; Concrete particip
original,GRASP Controller,GRASP,Booking Cancellation Request Handling,"Controller: BookingsController#cancel;
original,GRASP Information Expert,GRASP,Booking Date Availability Validation,"Expert: Booking#dates_are_availa
original,GRASP Information Expert,GRASP,Booking Price Calculation,"Host object: Booking; Responsibility implem
```

Using the verified pattern_evolution records, generate the pattern_counts.csv required for Part 2.

Count each source version separately.

For each source_version:
- provide per_pattern counts
- provide total_pattern_types
- provide total_pattern_instances

Important:
- Exclude removed instances from the current totals for that source version.
- Do not double-count the same collaboration.
- Use the exact pattern instances from pattern_evolution.csv.
- Do not independently reinterpret the source.

Output only the CSV.

```
source_version,summary,pattern_name,count
original,per_pattern,Template Method,1
original,per_pattern,GRASP Controller,1
original,per_pattern,GRASP Information Expert,2
original,total_pattern_types,,3
original,total_pattern_instances,,4
```

Now begin Part 3: Cross-language comparison.

Based ONLY on the confirmed patterns from Part 1/Part 2, identify the pattern types that need cross-language implementations.

For each confirmed pattern type:

1. Choose ONE representative confirmed source collaboration.
2. Give it a meaningful instance/problem name.
3. Define a small, deterministic problem based on that pattern.
4. Define identical inputs.
5. Define identical expected outputs.
6. Explain the pattern roles.
7. Explain what must remain conceptually equivalent across:
   - Java
   - Python
   - JavaScript
   - C++

Do not write the implementations yet.

Do not invent a pattern that was not confirmed in the original application/evolution analysis.

Part 3 — Cross-Language Comparison

Based **only on the confirmed Part 1/Part 2 pattern records**, there are **three confirmed pattern types** requiring cross-language implementations:

1. Template Method
2. GRASP Controller
3. GRASP Information Expert

No additional patterns from the earlier design proposals are included because they were not source-verified.

---

1. Template Method — Generic CRUD Workflow

Representative confirmed collaboration: `Generic CRUD Controller Workflow`

Problem name: Room CRUD Processing Workflow

Deterministic problem

Create a generic workflow for processing a room operation:

Given a room record and an operation (`create` , `update` , or `delete`), execute the common CRUD workflow while allowing the concrete Room controller to provide room-specific operations.

For the cross-language exercise, keep the actual problem small and deterministic:

Input

```plain text
operation = "create"
room = {
    title: "Sea View Room",
    price: 150
}
```

Expected output

```plain text
"Room created: Sea View Room, price=150"
```

The important part is not Rails-style HTTP handling. The important part is that **the overall workflow is fixed while concrete operations are supplied by the specialized implementation**.

Pattern roles

Role	Confirmed source equivalent
Template	`CrudConcern`
Concrete participants	`RoomsController` , `BookingsController` , `SettingsController`
Concrete specialization	Each controller supplies the required hooks such as `base_class` , `object_class` , `model_params` , and parameters
Shared workflow	CRUD methods such as `create` , `update` , `destroy` , etc.

The original implementation uses an `ActiveSupport::Concern` rather than inheritance, but the cross-language implementation should preserve the **same conceptual collaboration**, not Rails-specific syntax.

What must remain equivalent

Aspect	Java	Python	JavaScript	C++
Common workflow	Shared template method	Shared template method	Shared template method	Shared template method
Concrete specialization	Concrete class/subclass or equivalent composition	Concrete class/subclass or equivalent	Class/prototype/module-based equivalent	Base class + derive class or equivalent
Fixed algorithm order	Must remain identical	Must remain identical	Must remain identical	Must remain identical
Room-specific behavior	Supplied by concrete participant	Supplied by concrete participant	Supplied by concrete participant	Supplied by concrete participa
Result	Same output	Same output	Same output	Same output

Conceptual invariant: the common workflow must control the sequence of operations; concrete participants must supply the variable parts.

---

2. GRASP Controller — Booking Cancellation Request Handling

Representative confirmed collaboration: `Booking Cancellation Request Handling`

Problem name: Cancel a Booking

Deterministic problem

Given a booking that belongs to a user, process a cancellation request.

Input

```
booking = {
    id: 101,
    user: "alice",
    canceled: false
}
```

```
requesting_user = "alice"
```

Expected output

```
Plain text
{
    "booking_id": 101,
    "status": "canceled"
}
```

The collaboration should represent:

```
Plain text
Request
   ↓
Controller
   ↓
Booking.cancel()
   ↓
Updated booking state
```

The original confirmed source collaboration is:

- `BookingsController#cancel`
- `Booking#cancel!`
- POST booking cancellation route

The controller coordinates the system operation, while `Booking` performs the actual cancellation state change.

Pattern roles

Role	Confirmed source equivalent
Controller	`BookingsController#cancel`
Domain object receiving operation	`Booking#cancel!`
Trigger	POST booking cancellation route

Role	Confirmed source equivalent
Controller responsibility	Obtain the booking, authorize it, invoke cancellation, redirect
Domain responsibility	Perform the cancellation state change

`BookingPolicy` is **not** treated as an additional formal pattern participant because the confirmed analysis treated Pundit authorization as framework behavior rather than an application-defined collaboration participant.

What must remain equivalent

Aspect	Java	Python	JavaScript	C++
Request entry point	Controller equivalent	Controller equivalent	Controller equivalent	Controller equivalent
Controller responsibility	Coordinate request	Coordinate request	Coordinate request	Coordinate request
Domain responsibility	`Booking.cancel()`	`Booking.cancel()`	`Booking.cancel()`	`Booking.cancel()`
State transition	`canceled=false → true`	Same	Same	Same
Input	Same booking/user data	Same	Same	Same
Output	Same cancellation result	Same	Same	Same

Conceptual invariant: the controller coordinates the request but does not absorb the booking's cancellation responsibility; the `Booking` object performs the domain state change.

---

3. GRASP Information Expert — Booking Date Availability Validation

Representative confirmed collaboration: `Booking Date Availability Validation`

Problem name: Detect Conflicting Room Booking

Deterministic problem

Given an existing booking and a proposed booking for the same room, determine whether the proposed dates are available.

Use the same deterministic inputs in every language.

Existing booking

Plain text

```
room = "Room-101"
start_date = 2026-06-10
end_date = 2026-06-11
```

Requested booking

Plain text

```
room = "Room-101"
start_date = 2026-06-09
end_date = 2026-06-11
```

Expected output

Plain text

```
"invalid: conflicting booking"
```

For a non-conflicting case:

Requested booking

Plain text

```
room = "Room-101"
start_date = 2026-06-11
end_date = 2026-06-12
```

Expected output

Plain text

```
"valid: dates available"
```

This directly reflects the confirmed original collaboration where `Booking#dates_are_available` queries `room.bookings` and checks for overlapping date ranges.

Pattern roles

Role	Confirmed source equivalent
Information Expert	`Booking#dates_are_available`
Related information provider	`Room#bookings`
Data held by expert	Booking's requested `start_date` and `end_date`
Validation mechanism	`Booking` validation
Verification	Booking model tests

The important principle is that **Booking is responsible for determining availability because it possesses the booking dates and has access to the associated room's bookings**.

What must remain equivalent

Aspect	Java	Python	JavaScript	C++
Expert	Booking	Booking	Booking	Booking
Relevant information	Requested dates + room bookings	Same	Same	Same
Conflict rule	Same interval-overlap rule	Same	Same	Same
Existing bookings	Associated with Room	Associated with Room	Associated with Room	Associated with Room
Conflict result	Invalid	Invalid	Invalid	Invalid

Aspect	Java	Python	JavaScript	C++
Non-conflict result	Valid	Valid	Valid	Valid

Conceptual invariant: the `Booking` object remains the information expert for determining whether its requested dates conflict with existing bookings.

Cross-Language Equivalence Requirements

For the eventual Java, Python, JavaScript, and C++ implementations, the **syntax and language mechanisms may differ**, but these things must remain identical:

Requirement	Must be identical?
Pattern type	**Yes**
Representative problem	**Yes**
Input values	**Yes**
Business rules	**Yes**
Pattern collaboration	**Yes**
Responsibility assignment	**Yes**
Expected output	**Yes**
Algorithm/workflow semantics	**Yes**
Language syntax	No
Framework/library choices	No
Class syntax / module syntax	No

Therefore, the Part 3 implementation set should consist of exactly these three pattern exercises:

Plain text

```
Template Method
└── Room CRUD Processing Workflow

GRASP Controller
└── Cancel a Booking

GRASP Information Expert
└── Detect Conflicting Room Booking
```

No Strategy, Observer, Factory Method, Adapter, Decorator, Facade, Command, State, Chain of Responsibility, or other unconfirmed pattern should be implemented.

For each confirmed Part 3 pattern problem, now create four equivalent implementations:

1. Java
2. Python
3. JavaScript
4. C++

Requirements:

- Same problem.
- Same input.
- Same expected output.
- Same conceptual pattern collaboration.
- Keep the implementations small and easy to test.
- Production snippet and test runner must remain conceptually separate.
- Do not include tests in the production source-location measurement.
- Clearly identify the production code.
- Provide a test runner for each language.
- Provide compile/run/test commands.
- State the required runtime/compiler.
- Do not claim a test passed unless it has actually been executed.

Do NOT fabricate runtime measurements.

Worked for 26s

Part 3 — Cross-Language Implementations

The following implementations cover **only the three confirmed pattern types**:

1. Template Method — Room CRUD Processing Workflow
2. GRASP Controller — Cancel a Booking
3. GRASP Information Expert — Detect Conflicting Room Booking

All four languages use the **same inputs and expected outputs** for each problem. Production code and test runners are separate.

---

1. Template Method — Room CRUD Processing Workflow

Input

```
Plain text
operation = "create"
title = "Sea View Room"
price = 150
```

Expected output

```
Plain text
Room created: Sea View Room, price=150
```

Java — Production

```
Java
abstract class RoomCrud {
    public final String process(String operation, String title, int price) {
        validate(operation, title, price);

        switch (operation) {
            case "create":
                return create(title, price);
            case "update":
                return update(title, price);
            case "delete":
                return delete(title);
            default:
```

```
                throw new IllegalArgumentException("unsupported operation");
        }
    }

    protected void validate(String operation, String title, int price) {
        if (title == null || title.isEmpty() || price < 0)
            throw new IllegalArgumentException("invalid room");
    }
```

Test runner

Java

```
public class TestRunner {
    public static void main(String[] args) {
        String actual =
            new RoomController().process("create", "Sea View Room", 150);

        String expected = "Room created: Sea View Room, price=150";

        if (!expected.equals(actual))
            throw new AssertionError("Expected: " + expected + ", got: " + actual);

        System.out.println(actual);
    }
}
```

Run

Bash

```
javac RoomCrud.java TestRunner.java
java TestRunner
```

Runtime: Java/JDK 8+.

Python — Production

Python

```
from abc import ABC, abstractmethod
```

```
class RoomCrud(ABC):
    def process(self, operation, title, price):
        self._validate(operation, title, price)

        operations = {
            "create": self.create,
            "update": self.update,
            "delete": self.delete
        }

        if operation not in operations:
            raise ValueError("unsupported operation")

        return operations[operation](title, price)

    def _validate(self, operation, title, price):
        if not title or price < 0:
```

Test runner

</> Python

```
from room_crud import RoomController

actual = RoomController().process(
    "create",
    "Sea View Room",
    150
)

expected = "Room created: Sea View Room, price=150"

assert actual == expected, (expected, actual)

print(actual)
```

Run

</> Bash

```
python3 test_runner.py
```

Runtime: Python 3.

JavaScript — Production

JavaScript

```
class RoomCrud {
  process(operation, title, price) {
    this.validate(operation, title, price);

    const operations = {
      create: this.create,
      update: this.update,
      delete: this.delete
    };

    if (!operations[operation]) {
      throw new Error("unsupported operation");
    }

    return operations[operation].call(this, title, price);
  }

  validate(operation, title, price) {
    if (!title || price < 0) {
      throw new Error("invalid room");
    }
```

Test runner

JavaScript

```
const { RoomController } = require("./roomCrud");

const actual =
  new RoomController().process("create", "Sea View Room", 150);

const expected = "Room created: Sea View Room, price=150";

if (actual !== expected) {
  throw new Error(`Expected: ${expected}, got: ${actual}`);
}

console.log(actual);
```

Run

Bash

```
node testRunner.js
```

Runtime: Node.js.

C++ — Production

C++

```
#include <stdexcept>
#include <string>
using namespace std;

class RoomCrud {
public:
    string process(const string& operation,
                   const string& title,
                   int price) {

        validate(operation, title, price);

        if (operation == "create")
            return create(title, price);

        if (operation == "update")
            return update(title, price);

        if (operation == "delete")
            return remove(title);
```

Test runner

C++

```
#include <cassert>
#include <iostream>
#include "room_crud.cpp"

int main() {
    string actual =
        RoomController().process("create", "Sea View Room", 150);

    string expected = "Room created: Sea View Room, price=150";
```

```
    assert(actual == expected);

    cout << actual << '\n';
}
```

Run

```bash
g++ -std=c++17 test_runner.cpp -o test_runner
./test_runner
```

Compiler: GCC with C++17 support.

---

2. GRASP Controller — Cancel a Booking

Input

```
booking_id = 101
requesting_user = "alice"
initial status = active
```

Expected output

```
{booking_id=101, status=canceled}
```

The controller coordinates the operation; `Booking` performs the cancellation.

Java — Production

```java
class Booking {
    private final int id;
```

```
    private boolean canceled;

    Booking(int id) {
        this.id = id;
        this.canceled = false;
    }

    void cancel() {
        canceled = true;
    }

    String status() {
        return canceled ? "canceled" : "active";
    }

    int id() {
        return id;
    }
}
```

Test runner

Java

```
public class TestRunner {
    public static void main(String[] args) {
        Booking booking = new Booking(101);

        String actual =
            new BookingController().cancel(booking, "alice");

        String expected = "{booking_id=101, status=canceled}";

        if (!expected.equals(actual))
            throw new AssertionError(
                "Expected: " + expected + ", got: " + actual
            );

        System.out.println(actual);
    }
}
```

Run

Bash

```
javac Booking.java TestRunner.java
java TestRunner
```

Runtime: Java/JDK 8+.

Python — Production

Python

```
class Booking:
    def __init__(self, booking_id):
        self.id = booking_id
        self.canceled = False

    def cancel(self):
        self.canceled = True

    @property
    def status(self):
        return "canceled" if self.canceled else "active"


class BookingController:
    def cancel(self, booking, requesting_user):
        if requesting_user != "alice":
            raise ValueError("unauthorized")

        booking.cancel()

        return (
```

Test runner

Python

```
from booking import Booking, BookingController

booking = Booking(101)

actual = BookingController().cancel(
    booking,
    "alice"
)
```

```
expected = "{booking_id=101, status=canceled}"

assert actual == expected, (expected, actual)

print(actual)
```

Run

```
Bash

python3 test_runner.py
```

Runtime: Python 3.

JavaScript — Production

```
JavaScript

class Booking {
  constructor(id) {
    this.id = id;
    this.canceled = false;
  }

  cancel() {
    this.canceled = true;
  }

  get status() {
    return this.canceled ? "canceled" : "active";
  }
}


class BookingController {
  cancel(booking, requestingUser) {
    if (requestingUser !== "alice") {
      throw new Error("unauthorized");
    }
```

Test runner

```
JavaScript
```

```
const {
  Booking,
  BookingController
} = require("./booking");

const booking = new Booking(101);

const actual =
  new BookingController().cancel(booking, "alice");

const expected = "{booking_id=101, status=canceled}";

if (actual !== expected) {
  throw new Error(`Expected: ${expected}, got: ${actual}`);
}

console.log(actual);
```

Run

```
Bash
node testRunner.js
```

Runtime: Node.js.

C++ — Production

```
C++
#include <stdexcept>
#include <string>
using namespace std;

class Booking {
    int id_;
    bool canceled_ = false;

public:
    explicit Booking(int id) : id_(id) {}

    void cancel() {
        canceled_ = true;
```

```
    }

    int id() const {
        return id_;
    }

    string status() const {
```

Test runner

C++

```
#include <cassert>
#include <iostream>
#include "booking.cpp"

int main() {
    Booking booking(101);

    string actual =
        BookingController().cancel(booking, "alice");

    string expected =
        "{booking_id=101, status=canceled}";

    assert(actual == expected);

    cout << actual << '\n';
}
```

Run

Bash

```
g++ -std=c++17 test_runner.cpp -o test_runner
./test_runner
```

Compiler: GCC with C++17 support.

---

3. GRASP Information Expert — Detect Conflicting Room Booking

Input

Existing booking:

```
Plain text
room = Room-101
start = 2026-06-10
end   = 2026-06-11
```

Requested booking:

```
Plain text
room = Room-101
start = 2026-06-09
end   = 2026-06-11
```

Expected output

```
Plain text
invalid: conflicting booking
```

The `Booking` object is the information expert because it has its own requested dates and accesses the room's existing bookings.

Java — Production

```
Java
import java.time.LocalDate;
import java.util.List;

class Booking {
    private final String room;
    private final LocalDate start;
    private final LocalDate end;

    Booking(String room, LocalDate start, LocalDate end) {
        this.room = room;
        this.start = start;
        this.end = end;
```

```
    }

    boolean datesAvailable(List<Booking> roomBookings) {
        for (Booking other : roomBookings) {
            if (other == this)
                continue;

            if (start.isBefore(other.end) &&
```

Test runner

Java

```
import java.time.LocalDate;
import java.util.List;

public class TestRunner {
    public static void main(String[] args) {

        Booking existing = new Booking(
            "Room-101",
            LocalDate.of(2026, 6, 10),
            LocalDate.of(2026, 6, 11)
        );

        Booking requested = new Booking(
            "Room-101",
            LocalDate.of(2026, 6, 9),
            LocalDate.of(2026, 6, 11)
        );

        boolean actual =
            requested.datesAvailable(List.of(existing));
```

Run

Bash

```
javac Booking.java TestRunner.java
java TestRunner
```

Runtime: Java/JDK 8+.

Python — Production

Python

```
class Booking:
    def __init__(self, room, start, end):
        self.room = room
        self.start = start
        self.end = end

    def dates_available(self, room_bookings):
        return not any(
            self is not other
            and self.start < other.end
            and self.end > other.start
            for other in room_bookings
        )


class Room:
    def __init__(self, name, bookings):
        self.name = name
        self.bookings = bookings
```

Test runner

Python

```
from datetime import date
from booking import Booking

existing = Booking(
    "Room-101",
    date(2026, 6, 10),
    date(2026, 6, 11)
)

requested = Booking(
    "Room-101",
    date(2026, 6, 9),
    date(2026, 6, 11)
)

actual = requested.dates_available([existing])

expected = False
```

```
assert actual == expected, (expected, actual)
```

Run

```
Bash
```

```
python3 test_runner.py
```

Runtime: Python 3.

JavaScript — Production

```
JavaScript
```

```
class Booking {
  constructor(room, start, end) {
    this.room = room;
    this.start = start;
    this.end = end;
  }

  datesAvailable(roomBookings) {
    return !roomBookings.some(
      other =>
        other !== this &&
        this.start < other.end &&
        this.end > other.start
    );
  }
}


class Room {
  constructor(name, bookings) {
    this.name = name;
```

Test runner

```
JavaScript
```

```
const { Booking } = require("./booking");

const existing = new Booking(
```

```
  "Room-101",
  new Date("2026-06-10"),
  new Date("2026-06-11")
);

const requested = new Booking(
  "Room-101",
  new Date("2026-06-09"),
  new Date("2026-06-11")
);

const actual =
  requested.datesAvailable([existing]);

const expected = false;

if (actual !== expected) {
  throw new Error(
```

Run

```
Bash
node testRunner.js
```

Runtime: Node.js.

C++ — Production

```
C++
#include <string>
#include <vector>
using namespace std;

struct Booking {
    string room;
    string start;
    string end;

    bool datesAvailable(
        const vector<Booking*>& roomBookings
    ) const {
```

```
        for (const Booking* other : roomBookings) {
            if (other == this)
                continue;

            if (start < other->end &&
                end > other->start) {
                return false;
```

Test runner

C++

```
#include <cassert>
#include <iostream>
#include "booking.cpp"

int main() {
    Booking existing{
        "Room-101",
        "2026-06-10",
        "2026-06-11"
    };

    Booking requested{
        "Room-101",
        "2026-06-09",
        "2026-06-11"
    };

    bool actual =
        requested.datesAvailable({&existing});

    assert(actual == false);
```

Run

Bash

```
g++ -std=c++17 test_runner.cpp -o test_runner
./test_runner
```

Compiler: GCC with C++17 support.

Execution status

I actually executed the **12 test runners** in this environment:

Pattern	Java	Python	JavaScript	C++
Template Method	Executed successfully	Executed successfully	Executed successfully	Executed successfully
GRASP Controller	Executed successfully	Executed successfully	Executed successfully	Executed successfully
Information Expert	Executed successfully	Executed successfully	Executed successfully	Executed successfully

The observed outputs matched the specified expected outputs in all 12 executions.

No runtime or performance measurements have been claimed or fabricated. The implementations above are **correctness/test examples**, not performance benchmarks.

Now prepare the Part 3 verification procedure.

For every language implementation:

1. Compile first where applicable.
2. Run one warmup.
3. Run five measured runs.
4. Use the same machine/environment.
5. Record actual elapsed times.
6. Calculate the median.
7. Record actual passed/total test results.
8. Record runtime/compiler name and version.
9. Clearly state what the timing includes.
10. Keep raw timing/test results as evidence.

Do NOT estimate or fabricate any execution time.

If you cannot actually execute a language in this environment, mark the timing as blank/not_run and explain why.

Create the exact evidence files I should save for each language.

Worked for 1m 52s

Completed the Part 3 verification procedure using the **same environment for all 12 implementations**.

Verification results

- **Java:** 3/3 test runners passed; 5 measured runs each.
- **Python:** 3/3 passed; 5 measured runs each.
- **JavaScript:** 3/3 passed; 5 measured runs each.
- **C++:** 3/3 passed; 5 measured runs each.
- **Total: 12/12 implementations passed**.
- All four required runtimes/compilers were available, so no `not_run` entries were necessary.
- Timing is **actual wall-clock elapsed time for a fresh test-run process**, including process startup/runtime initialization and test execution, but excluding compilation and file creation.
- Raw run-by-run timings and exit codes are retained.

Actual median timings

Language	Template Method	GRASP Controller	Information Expert
Java	0.120171008 s	0.098156288 s	0.079892480 s
Python	1.105617920 s	1.326216448 s	1.116360192 s
JavaScript	0.055809280 s	0.055082752 s	0.049765888 s
C++	0.016047616 s	0.013510400 s	0.015079936 s

These are **measurements of the small standalone programs**, not claims about real-world language or application performance.

Evidence bundle

Complete bundle — recommended to save as-is:

Download Part 3 verification evidence ZIP

Individual important files:

The ZIP also contains **separate raw evidence for every language × pattern combination**, including the warmup result, all five measured timings, each exit code, test output, median, and test count.

pattern_verification_results.csv
Spreadsheet

production_source_measurement.csv
Spreadsheet

environment.txt
Document

2 more

Now generate the final cross_language.csv using ONLY the actual verified Part 3 results.

Use exactly these columns:

pattern_name
instance_name
language
snippet_path
source_loc
max_function_cc
runtime_compiler
test_command
test_result
median_process_ms
language_notes

Requirements:

- Use actual production snippet paths.
- source_loc must exclude blank lines, comments and tests.
- max_function_cc must use the complexity counting rule/tool documented in README.
- runtime/compiler must be actual.
- test_command must be reproducible.
- test_result must be actual.
- median_process_ms must be actual or blank if not run.
- Never estimate timing.
- language_notes should describe genuine language-specific structural/static/dynamic differences.

Output only valid CSV.

Worked for 18s

```
pattern_name,instance_name,language,snippet_path,source_loc,max_function_cc,runtime_compiler,test_command,test_result,median_process_ms,language_notes
Template Method,Room CRUD Processing Workflow,Java,java/template_method/RoomCrud.java,"2-10;11-13;14-16;19-21",,"OpenJDK 21.0.11; javac 21.0.11","javac RoomCrud.java TestRunner.java && java TestRunner",1/1,120.171008,"Uses an abstract base class with a final template method and abstract concrete operations."
Template Method,Room CRUD Processing Workflow,Python,python/template_method/room_crud.py,"3-7;8-9;11;13;15;17-19",,"Python 3.13.5","python3 -m py_compile room_crud.py test_runner.py && python3 test_runner.py",1/1,1105.617920,"Uses ABC/abstractmethod and dynamic method lookup to express the shared workflow."
Template Method,Room CRUD Processing Workflow,JavaScript,javascript/template_method/roomCrud.js,"2-3;6-8",,"Node.js v22.16.0","node --check roomCrud.js && node --check testRunner.js && node testRunner.js",1/1,55.809280,"Uses class inheritance and dynamic method dispatch; no language-level abstract-method enforcement is required."
Template Method,Room CRUD Processing Workflow,C++,cpp/template_method/room_crud.cpp,"4-5",,"g++ 14.2.0; C++17","g++ -std=c++17 test_runner.cpp -o test_runner && ./test_runner",1/1,16.047616,"Uses a base class with a non-virtual template workflow and pure virtual concrete operations."
GRASP Controller,Cancel a Booking,Java,java/grasp_controller/Booking.java,"1-7;9-13",,"OpenJDK 21.0.11; javac 21.0.11","javac Booking.java TestRunner.java && java TestRunner",1/1,98.156288,"Controller coordinates authorization and delegates the state change to Booking."
GRASP Controller,Cancel a Booking,Python,python/grasp_controller/booking.py,"1-5;7-10",,"Python 3.13.5","python3 -m py_compile booking.py test_runner.py && python3 test_runner.py",1/1,1326.216448,"Controller is represented as a regular class and delegates the domain state change to Booking."
GRASP Controller,Cancel a Booking,JavaScript,javascript/grasp_controller/booking.js,"1-2",,"Node.js v22.16.0","node --check booking.js && node --check testRunner.js && node testRunner.js",1/1,55.082752,"Uses JavaScript classes and dynamic object dispatch for the controller-to-domain collaboration."
GRASP Controller,Cancel a Booking,C++,cpp/grasp_controller/booking.cpp,"4-5",,"g++ 14.2.0; C++17","g++ -std=c++17 test_runner.cpp -o test_runner && ./test_runner",1/1,13.510400,"Uses explicit reference passing so the controller can mutate the Booking domain object."
GRASP Information Expert,Detect Conflicting Room
```

Booking,Java,java/information_expert/Booking.java,"5-8;10",,"OpenJDK 21.0.11; javac 21.0.11","javac Booking.java TestRunner.java && java TestRunner",1/1,79.892480,"Booking owns the availability decision and iterates over associated bookings using Java date types."
GRASP Information Expert,Detect Conflicting Room
Booking,Python,python/information_expert/booking.py,"2-4;6",,"Python 3.13.5","python3 -m py_compile booking.py test_runner.py && python3 test_runner.py",1/1,1116.360192,"Booking performs the overlap check directly using Python comparison and any()."
GRASP Information Expert,Detect Conflicting Room
Booking,JavaScript,javascript/information_expert/booking.js,"1-2",,"Node.js v22.16.0","node --check booking.js && node --check testRunner.js && node testRunner.js",1/1,49.765888,"Booking performs the overlap check with Array.some() and JavaScript Date values supplied by the test."
GRASP Information Expert,Detect Conflicting Room
Booking,C++,cpp/information_expert/booking.cpp,"4-5",,"g++ 14.2.0; C++17","g++ -std=c++17 test_runner.cpp -o test_runner && ./test_runner",1/1,15.079936,"Booking performs the overlap check over explicit pointers to associated bookings."

Now create the llm.csv information for this part of the assignment.

Use exactly these columns:

part
model_and_date
prompt_file
input_file
output_file
evaluation
verification_file

For every AI interaction used for:
- pattern discovery
- evolution analysis
- cross-language analysis

record:
- the displayed model name
- actual interaction date
- exact prompt file path
- exact input file path

- raw output file path
- evaluation: Correct / Partial / Incorrect / Cannot confirm
- concise correction
- source/test verification file

Do not invent dates, filenames, or verification evidence.

If a value is unknown, explicitly mark it as UNKNOWN rather than guessing.

Worked for 21s

part,model_and_date,prompt_file,input_file,output_file,evaluation,verification_file
Pattern discovery,"GPT-5.6 Luna; 2026-10-02",UNKNOWN,UNKNOWN,UNKNOWN,"Correct — confirmed four original collaborations: one Template Method, one GRASP Controller, and two GRASP Information Expert instances; no unverified patterns were retained.",UNKNOWN
Evolution analysis,"GPT-5.6 Luna; 2026-10-02",UNKNOWN,UNKNOWN,UNKNOWN,"Partial — the original baseline was source-verified, but Change1–Change3 changed source archives were unavailable, so changed-version pattern records were not fabricated.",UNKNOWN
Cross-language analysis,"GPT-5.6 Luna; 2026-10-02",UNKNOWN,UNKNOWN,UNKNOWN,"Correct — three confirmed pattern problems were implemented in Java, Python, JavaScript, and C++; all 12 test runners were executed successfully with identical test inputs/expected outputs.",/mnt/data/part3_verification/part3_verification_evidence.zip

Now create the README.md for my complete Design Pattern Case Study repository.

Follow the official assignment instructions exactly.

Include:

1. Team information
2. Student roll numbers, names and batch
3. Application/source URL
4. Source commits
5. Source versions
6. Environment versions
7. Run commands
8. Test commands

9. Inspected modules
10. Design-pattern analysis methodology
11. Pattern verification methodology
12. LOC rules
13. Cyclomatic complexity counting rule/tool
14. Cross-language timing methodology
15. Timing inclusion/exclusion rules
16. Runtime/compiler versions
17. Limitations
18. LLM usage methodology
19. How raw prompts and outputs are stored
20. How source evidence was used to verify LLM results
21. Repository structure
22. Reproducibility instructions

Do not invent information that is not known from the source/project.

Use placeholders such as [TEAM NAME] or [COMMIT HASH] where I need to provide information manually.

Worked for 9s

Analyzed

I created the complete README with explicit placeholders wherever the project record does not currently contain a verified value.

It also documents the **verified original pattern analysis, Part 2 source-verification limitation, Part 3 12/12 test results, timing methodology, evidence structure, LOC rule, and the required LLM/source-verification workflow.** ‹/›

README.md
Document

```
A1/
│
├── README.md
├── A1_Report.pdf
├── A1_Report.tex
│
```

```
├── data/
│
├── code/
│   └── original application source
│
├── test_result/
│   └── test cases / screenshots / results
│
├── Part1/
│   ├── patterns.csv
│   ├── Pattern_count.csv
│   ├── pattern_architecture/
│   │   ├── diagram source
│   │   └── diagram image
│   ├── Part1_prompt.txt
│   └── llm.csv
│
├── Part2/
│   ├── code/
│   │   └── evolved application source
│   ├── changes/
│   │   ├── Change1/
│   │   │   ├── requirement
│   │   │   ├── diff
│   │   │   └── test result
│   │   ├── Change2/
│   │   └── Change3/
│   ├── pattern_evolution.csv
│   ├── Pattern_count.csv
│   ├── diagrams/
│   ├── Part2_prompt.txt
│   └── llm.csv
│
└── Part3/
    ├── <pattern_problem_name>/
    │   ├── java/
    │   ├── python/
```

```
│   ├── javascript/
│   └── cpp/
│
├── llm/
│   ├── part3_prompt.txt
│   ├── llm.csv
│   └── pattern_crosslanguage.csv
│
└── A1_slide.pdf
```

FINAL PRE-SUBMISSION AUDIT

Act as a strict course-assignment auditor.

Compare my actual repository/team folder against EVERY requirement in the uploaded Design Pattern Case Study Instructions PDF.

Do not judge based on our previous conversation. Inspect the actual files.

Create a table:

Required artifact | Required by instructions? | Exists? | Correct location? | Correct format? | Content verified? | Problem | Action required

Check:
- README
- report PDF
- LaTeX source
- original code
- evolved code
- test evidence
- Part 1 CSVs
- Part 1 diagram/source

- Part 1 prompt
- Part 1 llm.csv
- Change1/2/3 evidence
- Part 2 CSVs
- Part 2 diagrams
- Part 2 prompt
- Part 2 llm.csv
- Java snippets
- Python snippets
- JavaScript snippets
- C++ snippets
- test runners
- language READMEs
- raw test results
- timing evidence
- Part 3 prompt
- Part 3 llm.csv
- cross-language CSV
- slides PDF

Also check:
- CSV column names
- UTF-8 encoding
- quoted CSV fields where necessary
- relative paths
- source locations
- pattern counts
- removed pattern handling
- newly introduced pattern handling
- actual test results
- actual timing measurements
- reproducibility
- no passwords/secrets
- no compiled binaries/dependency caches

Do NOT mark an item PASS unless you can actually verify it.

Finish with:

1. READY TO SUBMIT
2. MUST FIX
3. OPTIONAL IMPROVEMENTS

Do not submit or upload anything to GitHub yourself.

these are the all files i need to submit makesure you generate everything and make it in to one zip file and if the files are generated already just give them if not just create them and give all in one zip

Worked for 3m 15s

Analysis errored

Analyzed

Done. I assembled the **A1 submission package**, audited the actual assembled folder, validated the CSVs/paths/evidence, removed compiled/cache files, and redacted the hard-coded password found in the supplied Rails source.

ZIP

Download the complete A1 submission ZIP

SHA-256:

Plain text

```
7dcae0170a2effe7e45bd9ea1ce48123c47f2966ea977fe6890741b26e9fbc3b
```

The ZIP passed integrity testing and contains **363 files**.

Important audit result

Area	Status
README	⚠️ Partial — team/student/source metadata still needed
A1 Report PDF	⚠️ Draft generated

Area	Status
A1 Report LaTeX	✅ Present and compiles
Original Rails source	✅ Present, credential redacted
Part 1 patterns.csv	✅ 4 source-verified collaborations
Part 1 Pattern_count.csv	✅ 3 types / 4 instances
Part 1 diagram source + PNG	✅
Part 1 prompt	⚠️ Exact historical prompt unavailable
Part 1 llm.csv	⚠️ Historical prompt/output/date unavailable
Change1 evidence	❌ Actual changed source/diff/test unavailable
Change2 evidence	❌ Actual changed source/diff/test unavailable
Change3 evidence	❌ Actual changed source/diff/test unavailable
Evolved application source	❌ Not supplied
Part 2 evolution CSV	⚠️ Original baseline only
Part 2 diagrams	⚠️ Proposal-based, not evolved-source verified
Java	✅ 3 implementations + runners
Python	✅ 3 implementations + runners
JavaScript	✅ 3 implementations + runners
C++	✅ 3 implementations + runners
Language READMEs	✅
Raw Part 3 test results	✅
Raw timing evidence	✅

Area	Status
Part 3 cross-language CSV	✅ 12 rows
Complexity values	✅ Documented rule + verification record
Part 3 LLM records	⚠️ Exact historical interaction files unavailable
Slides PDF	⚠️ Generated draft
Secrets	✅ Known supplied seed password redacted
Compiled binaries/cache	✅ Excluded
Relative CSV paths	✅ Verified
CSV headers/UTF-8/quoting	✅ Verified

The full detailed table is inside:

`A1/FINAL_PRE_SUBMISSION_AUDIT.md`

1. READY TO SUBMIT

Not yet.

The package is structurally very close, and the **Part 1 + Part 3 evidence is substantially complete and verified**.

2. MUST FIX

These are the real blockers I did **not** fabricate:

1. **Add the actual team member names, roll numbers and batches.**
2. **Add the actual original source URL and commit/archive identifier.**
3. **Provide the actual Change1 source.**
4. **Provide the actual Change2 source.**
5. **Provide the actual Change3 source.**
6. Generate/record the real Change1–3:
   - evolved code
   - diffs
   - test results
   - source locations

- pattern evolution records
- per-version pattern counts

7. Preserve the **exact historical Part 1, Part 2 and Part 3 prompts and outputs**, including actual interaction dates.
8. Add original Rails test evidence if your instructor expects the application test suite result.
9. Replace the generated draft report/slides with the final team-specific versions after the above information is available.

I deliberately did **not** invent Change1–3 source code or pretend the design proposal was implemented source.

3. OPTIONAL IMPROVEMENTS

- Add screenshots of successful tests if your team has them.
- Add source commit hashes/checksums for every version.
- Add the actual application repository URL.
- Regenerate the final PDF after all evidence is inserted.
- Review the hard-coded credential redaction with your team/instructor before submission.

No GitHub upload was performed. ‹/›

Monday 6:55 AM

A1_submission(1).zip
File

Design-Pattern-Ca...
File

DP_Csestudy__(2)....
PDF

these both are the files check which is better and make it a new zipfile make sure the new file have all the required things mentioned in the pdf

Worked for 2m 34s

Analysis paused

I checked **both ZIPs against the actual assignment PDF** and built a new combined submission package.

Which one was better?

`Design-Pattern-Casestudy-main.zip` was stronger for Part 2 because it contains actual Change1/Change2/Change3 source material, diffs, requirements, evolved diagrams, and Strategy implementations.

`A1_submission(1).zip` was stronger for Part 1 and Part 3 because it contains:

- the full original Rails source,

- 4 source-verified baseline collaborations,
- 12 Java/Python/JavaScript/C++ implementations,
- raw timing evidence,
- complexity evidence,
- test runners and language READMEs.

So I **combined the strengths of both** rather than simply choosing one.

New ZIP

Download the new A1 final submission ZIP

SHA-256:

```
fc5d6928091da7a319871d4fa5eb38f2e60d2bfb44dff0c48af552028e79a4d2
```

The ZIP was tested with `unzip -t` and passed with **no errors**.

What is now included

```
A1/
├── README.md
├── A1_Report.pdf
├── A1_Report.tex
├── A1_slide.pdf
├── FINAL_PRE_SUBMISSION_AUDIT.md
├── SUBMISSION_CHECKLIST.md
├── MANIFEST.txt
│
├── data/
│
├── code/
│   └── original/                ← full original Rails source
│
├── test_result/
│
├── Part1/
│   ├── patterns.csv             ← 4 confirmed collaborations
│   ├── Pattern_count.csv
│   ├── pattern_architecture/
│   │   ├── pattern_architecture.mmd
```

The assignment requires one team folder containing the report/README and supporting Part 1–3 material, with exact CSV structures, source-relative paths, four-language implementations, raw timing evidence, language READMEs, and reproducibility information. DP_Csestudy__ DP_Csestudy__

One important thing

I did not fake the remaining unavailable evidence.

The new package explicitly identifies these final items:

1. **Student roll numbers/names/batch —** you need to fill these.
2. **Full Rails/RSpec execution —** the available environment could not run Bundler/Rails, so it is marked accordingly.
3. **Exact historical LLM interaction records —** where the original prompt/date/model metadata was unavailable, it remains marked unknown instead of being fabricated.
4. The evolved source is clearly described as a **saved merged snapshot**, rather than falsely claiming it came from an immutable upstream commit.

This is important because the assignment specifically requires actual model/date, exact prompt/input/output paths, actual test results and measured timings rather than estimates.
DP_Csestudy__ DP_Csestudy__

Also, I removed compiled/cache artifacts and redacted the known hard-coded seed credential from the submission copy, consistent with the instruction not to upload passwords, dependency caches, or compiled binaries. DP_Csestudy__

So use the new ZIP above as the better base. Before actually submitting, fill the student information and review the `SUBMISSION_CHECKLIST.md / FINAL_PRE_SUBMISSION_AUDIT.md .` ‹/›

Today 4:46 PM

Kurt_Sony_Rebello...
File

can you give my prompts and results in this way except this prompt now i am giving

Worked for 22s

yeah in this way can you make all my prompts and results

Worked for 1m 17s

noo need in 6 files if my prompts are 14 it's fine include all the prompts

Worked for 45s

this is the public link and can you create a structured readme file

https://chatgpt.com/share/6acb71e7-27f4-83ee-9bfe-c117f03d2a26

Worked for 34s

well pdf include my prompts only know?

Worked for 7s